

Introducción teórico-práctica al Aprendizaje por Refuerzo

Clase 5— Monte Carlo & gym

Profesor: Matias Bilkis



UBA

Universidad de Buenos Aires



Antes de arrancar
Un par de comentarios











**TRAS EL FRACASO DE LA PARITARIA,
DOCENTES DE LA UBA ANUNCIARON UNA
NUEVA MARCHA Y 23 DÍAS DE PARO**

**a24noticias**  Follow ...

**a24noticias**  1h

 **TRAS EL FRACASO DE LA PARITARIA,
DOCENTES DE LA UBA ANUNCIARON
UNA NUEVA MARCHA Y 23 DÍAS DE PARO**

Los gremios Aduba y Apuba decretaron un paro escalonado y rotativo que se extenderá desde el 21 de septiembre hasta el 22 de octubre, tras no llegar a un acuerdo con el Gobierno nacional en la última negociación paritaria.

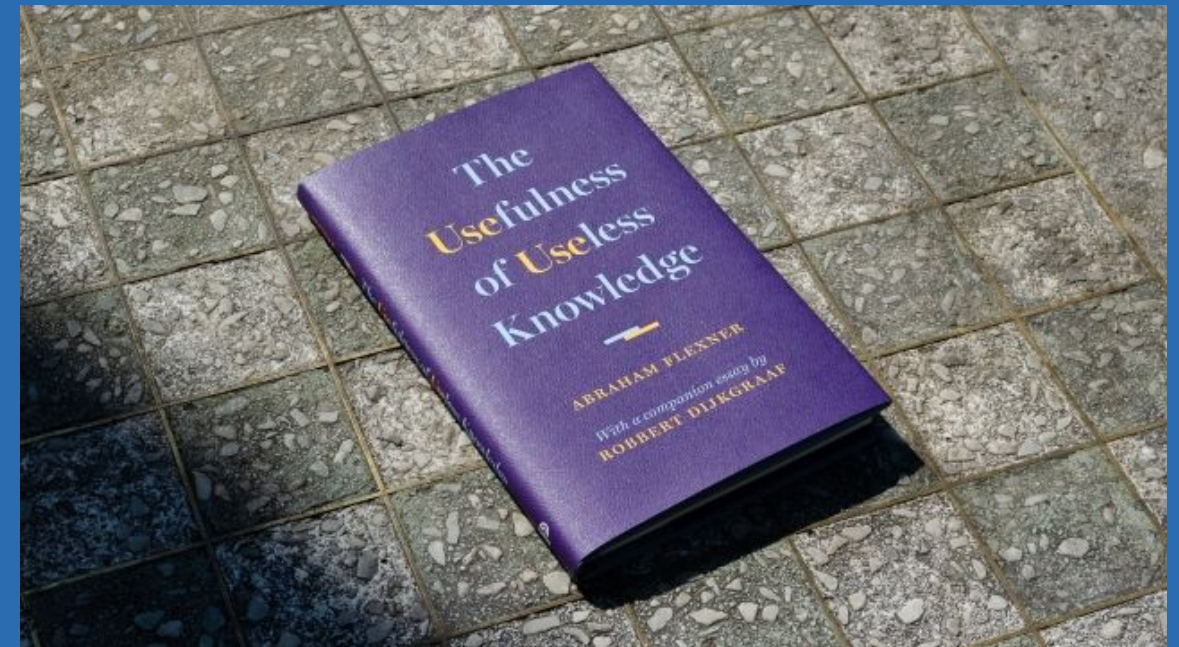
En el marco del plan de lucha por la aplicación plena de la ley de Financiamiento Universitario, las organizaciones convocaron además a la Quinta Marcha Federal Universitaria para el jueves 15 de octubre frente al Palacio de Tribunales.

Desde los sindicatos cuestionaron duramente el impacto de la

 116  30  4 

1 hour ago

 Add a comment... 





Mesa redonda

El impacto de la IA en la investigación científica

Pablo Groisman (DM-IMAS)

Gabriel Mindlin (DF-INFINA)

Enzo Ferrante (ICC)

Laura Kamenetzky (DFBMC-IB3)

Noosh Arrar (IC)

Juan Ruiz (DCAO y CIMA)

Modera: Julián Fernández Bonder

Viernes 18 de septiembre

16.00 hs | Aula 1401, Cero+Infinito

.UBA EXACTAS

¿Qué pasó con la
tarea Clase 4?

¿Cómo es la complejidad de VI?

$$\text{costo} \sim |\mathcal{S}|^2 \times |\mathcal{A}| \times \frac{\log \frac{R_{\max}}{(1-\gamma)\epsilon}}{\log 1/\gamma}$$

- 1) Contracción Banach $\|V_0 - V_*\| < \gamma^k R / (1-\gamma)$
- 2) Los loops
- 3) Gridworld puede ser mejor
- 4) Con paredes puede costar “propagar”
- 5) El scaling debe ser $O(|\mathcal{S}|) = O(N^2)$

Value Iteration, for estimating $\pi \approx \pi_*$

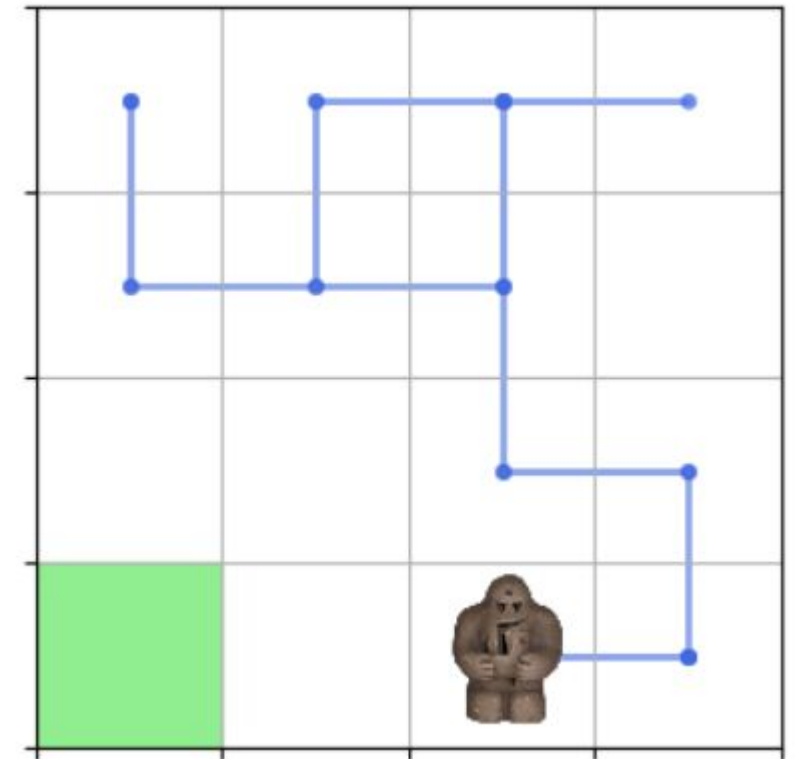
Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation
Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(\text{terminal}) = 0$

Loop:

$\Delta \leftarrow 0$
Loop for each $s \in \mathcal{S}$:
 $v \leftarrow V(s)$
 $V(s) \leftarrow \max_a \sum_{s',r} p(s',r|s,a) [r + \gamma V(s')]$
 $\Delta \leftarrow \max(\Delta, |v - V(s)|)$
until $\Delta < \theta$

Output a deterministic policy, $\pi \approx \pi_*$, such that
 $\pi(s) = \operatorname{argmax}_a \sum_{s',r} p(s',r|s,a) [r + \gamma V(s')]$

$\mathcal{S}$
Contracción



Monte Carlo: una motivación



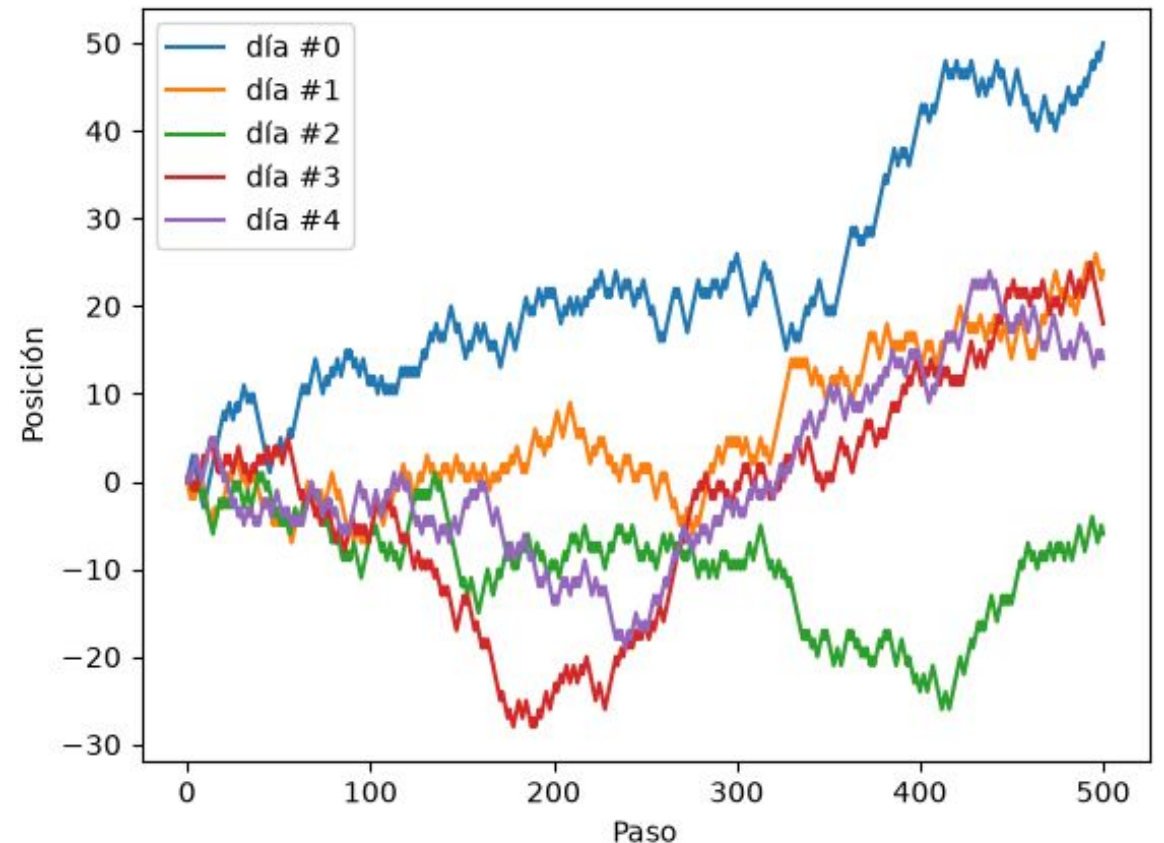
Tenemos un random walker

Consideramos un random-walk en 1D. En este caso, podemos calcular analíticamente diferentes cantidades, ejemplo, la distribución de probabilidad:

$$x_{t+1} = x_t + \xi_t, \quad x_0 = 0,$$

$$x_t = \sum_{k=0}^{t-1} \xi_k \quad \xi_t = \begin{cases} +1 & \text{con prob. } \frac{1}{2}, \\ -1 & \text{con prob. } \frac{1}{2}. \end{cases}$$

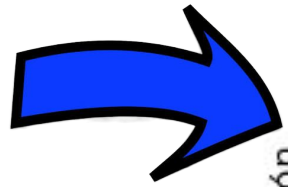
$$P(x_t = x) = \frac{1}{2^t} \binom{t}{\frac{t+x}{2}}, \quad |x| \leq t,$$



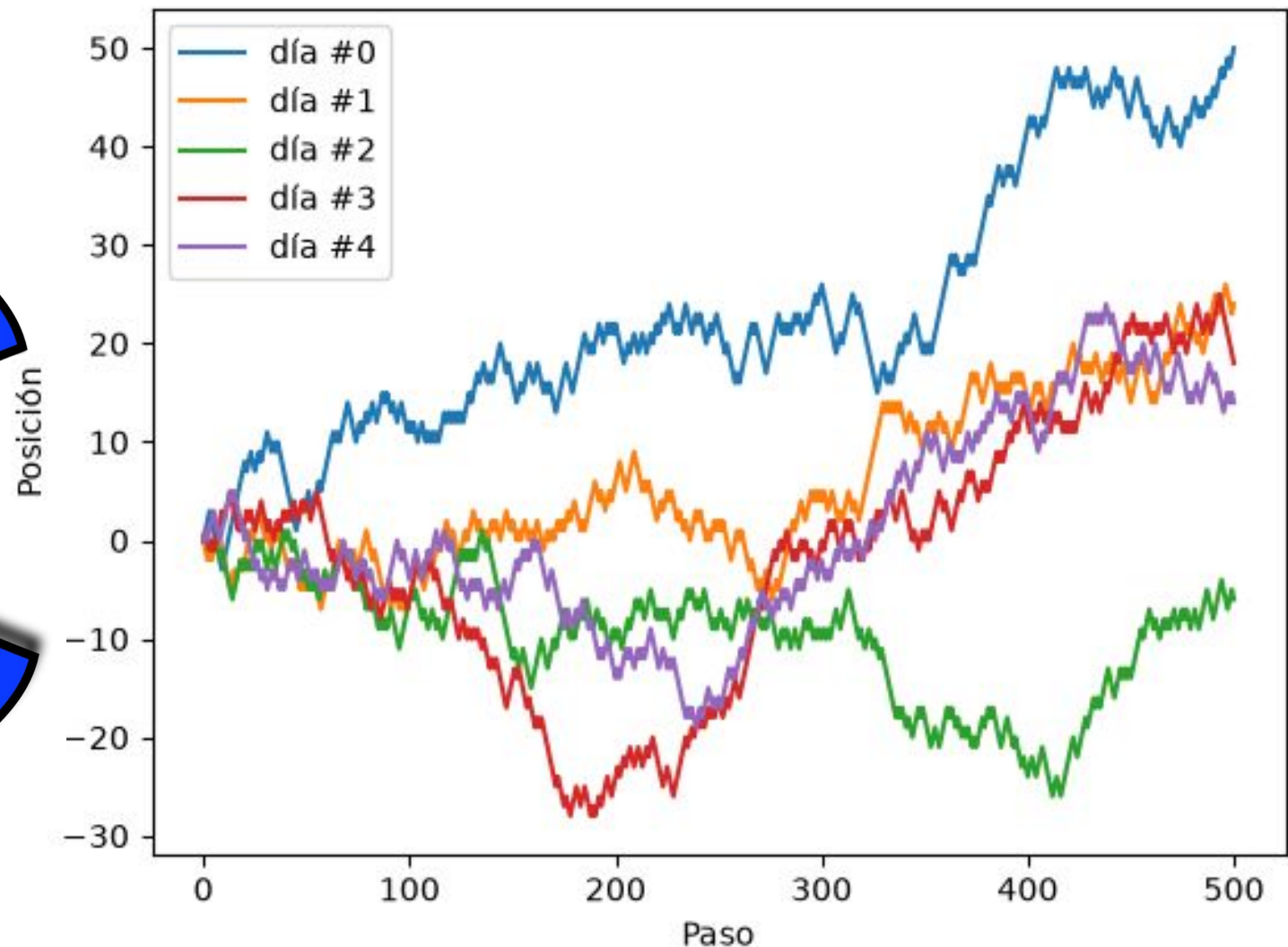
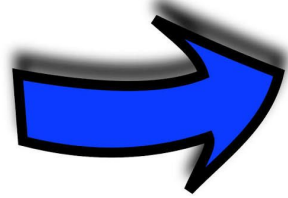
Tenemos un random walker

¿Pero qué pasa si empezamos a hacer preguntas del tipo “¿cuánto tardamos en llegar al súper sin pasar x control?”

#13



#4

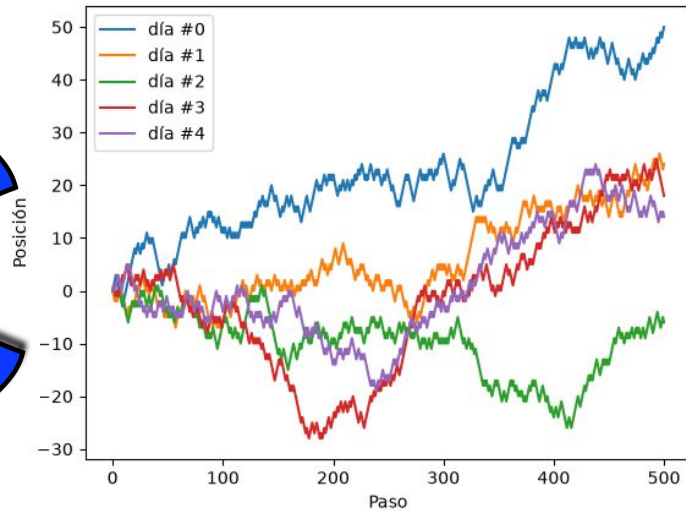
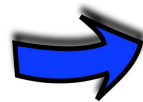
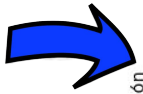


Tenemos un random walker

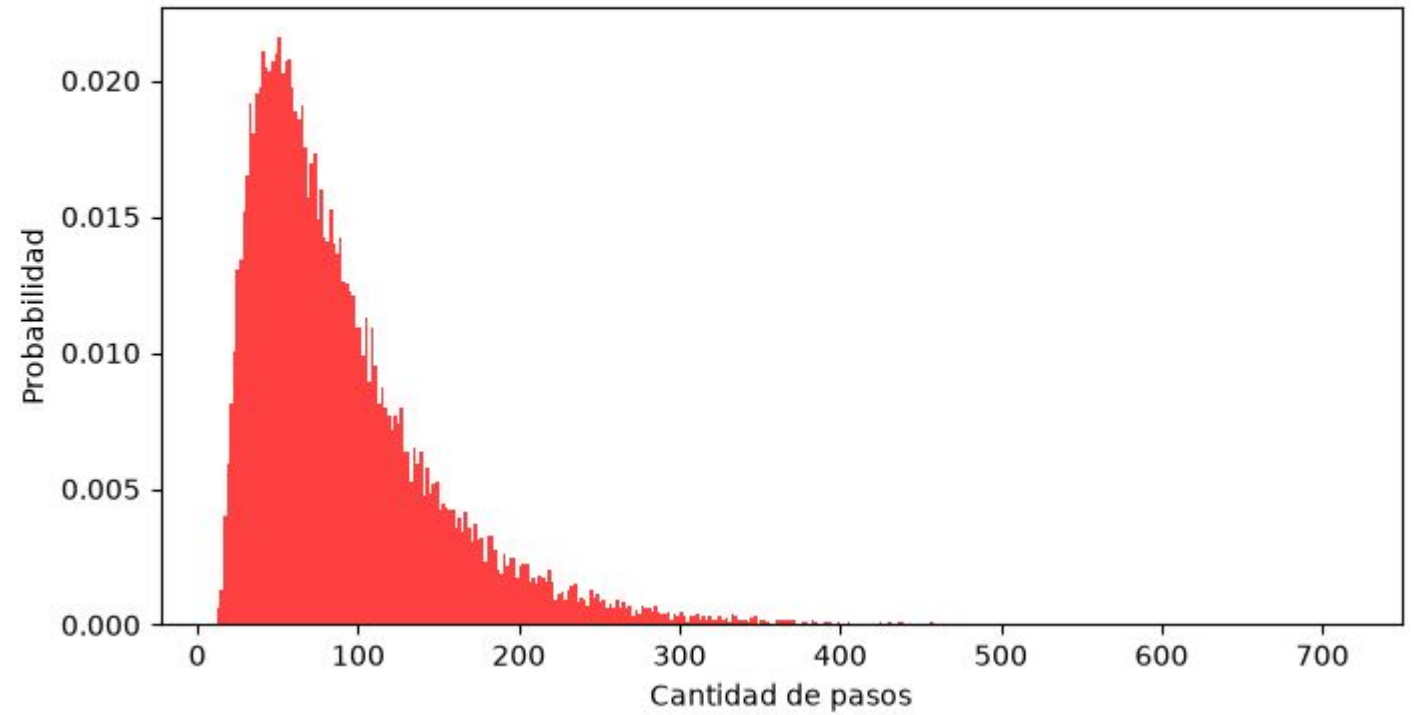
¿Pero qué pasa si empezamos a hacer preguntas del tipo “¿cuánto tardamos en llegar al súper sin pasar x control?”



Carrefour

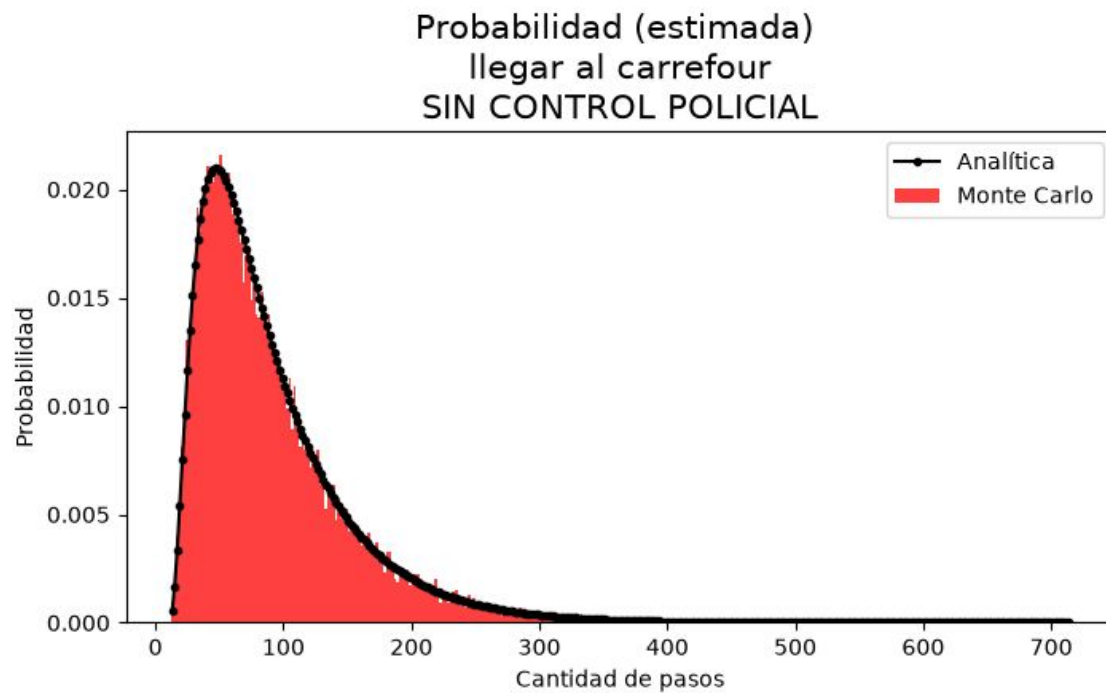


Probabilidad (estimada)
llegar al carrefour
SIN CONTROL POLICIAL



Tenemos un random walker

No es que no se pueda calcular (para este caso) la distribución de “llegar a carrefour sin pasar por control”...



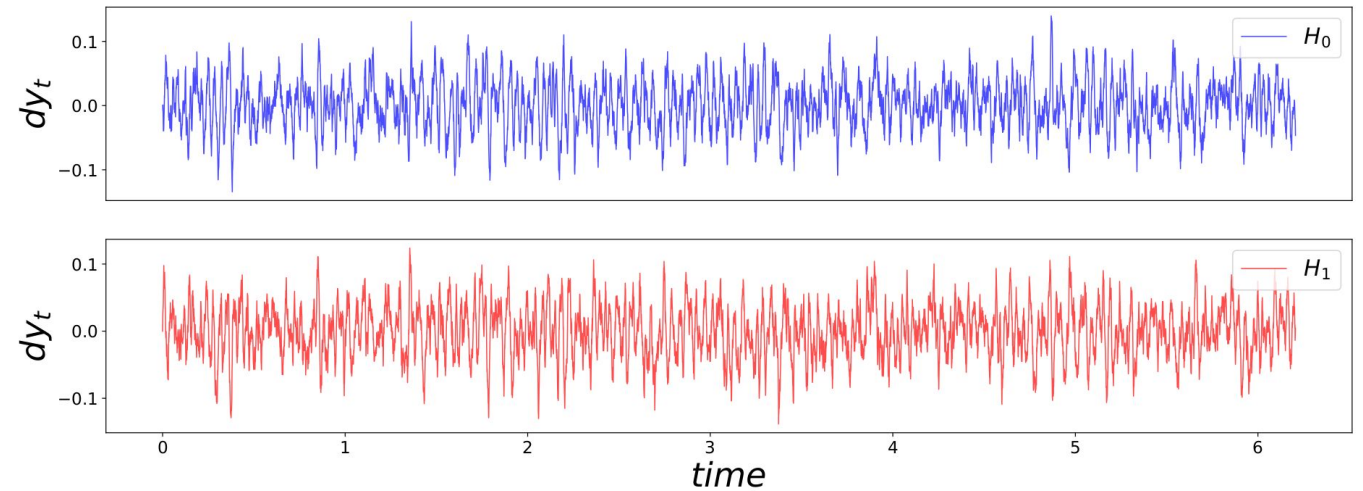
```
def random_walk_window():  
    x = steps = 0  
    while -4 < x < 13:  
        x += random.choice((-1, 1))  
        steps += 1  
    return steps if x == 13 else None
```

$$P(\tau_{13} = n \mid \tau_{13} < \tau_{-4}) = \frac{1}{4} \sum_{k=1}^{16} \sin\left(\frac{4k\pi}{17}\right) \sin\left(\frac{16k\pi}{17}\right) \cos^{n-1}\left(\frac{k\pi}{17}\right)$$

- (1) *bastante práctico,*
- (2) *un ejemplo de cómo se calcula con MC*

Rápidamente MC fundamental

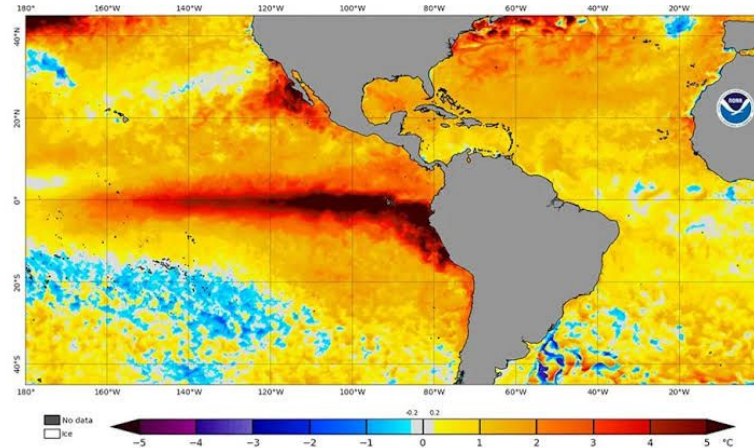
$$X_{t+1} = X_t + \varepsilon_t, \quad \varepsilon_t \sim \mathcal{N}(0, 1)$$



[\[SequentialQuantum\]](#)

¿Cuándo es útil Monte Carlo?

En general MC es fundamental cuando tenemos sistemas aleatorios que tienen muchas variables, y necesitamos predecir cosas.



Monte Carlo: ¿de qué se trata?

Motivación

*Dado el modelo... calculame algo...
usá tu PC...*



Stanisław Ulam



$$I_f = \mathbb{E}_\pi[f] = \begin{cases} \sum_{x \in \mathcal{S}} \pi(x) f(x) \\ \int dx f(x) \pi(x) \end{cases}$$

$\{x_1, \dots, x_k, \dots, x_N\}$



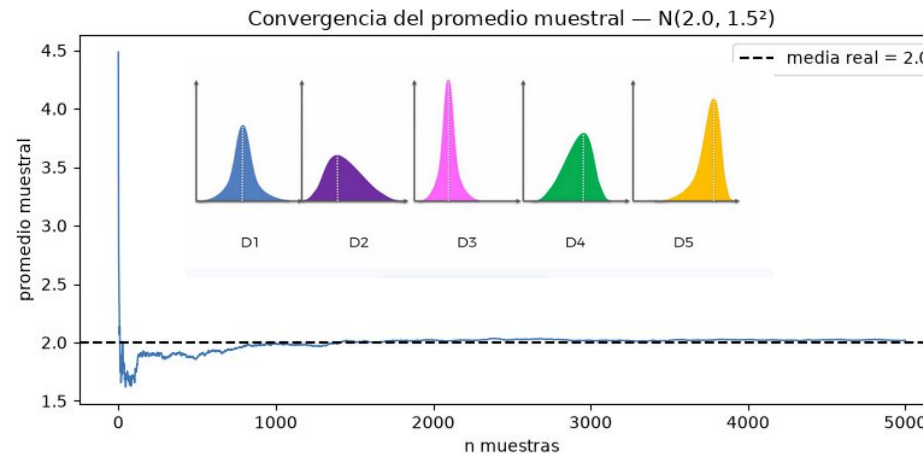
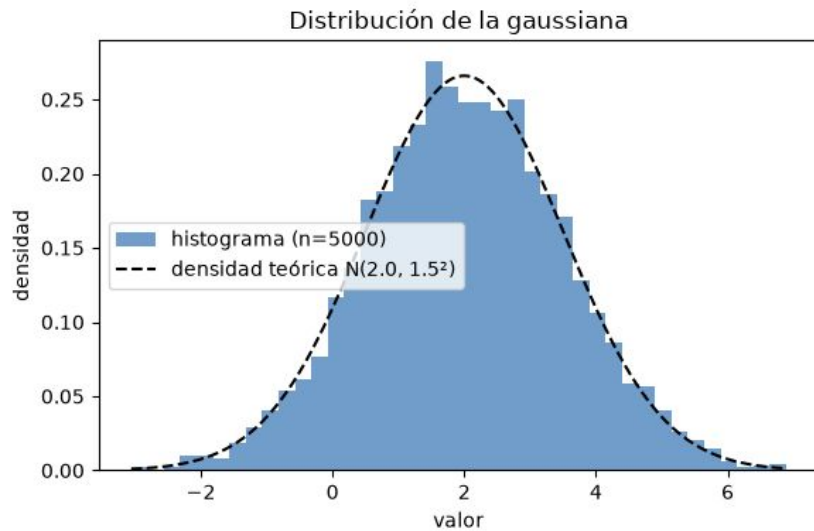
$$I \equiv \frac{1}{N} \sum_k f(x_k)$$

Motivación

Dado el modelo... calculame algo... usá PC



Stanisław Ulam



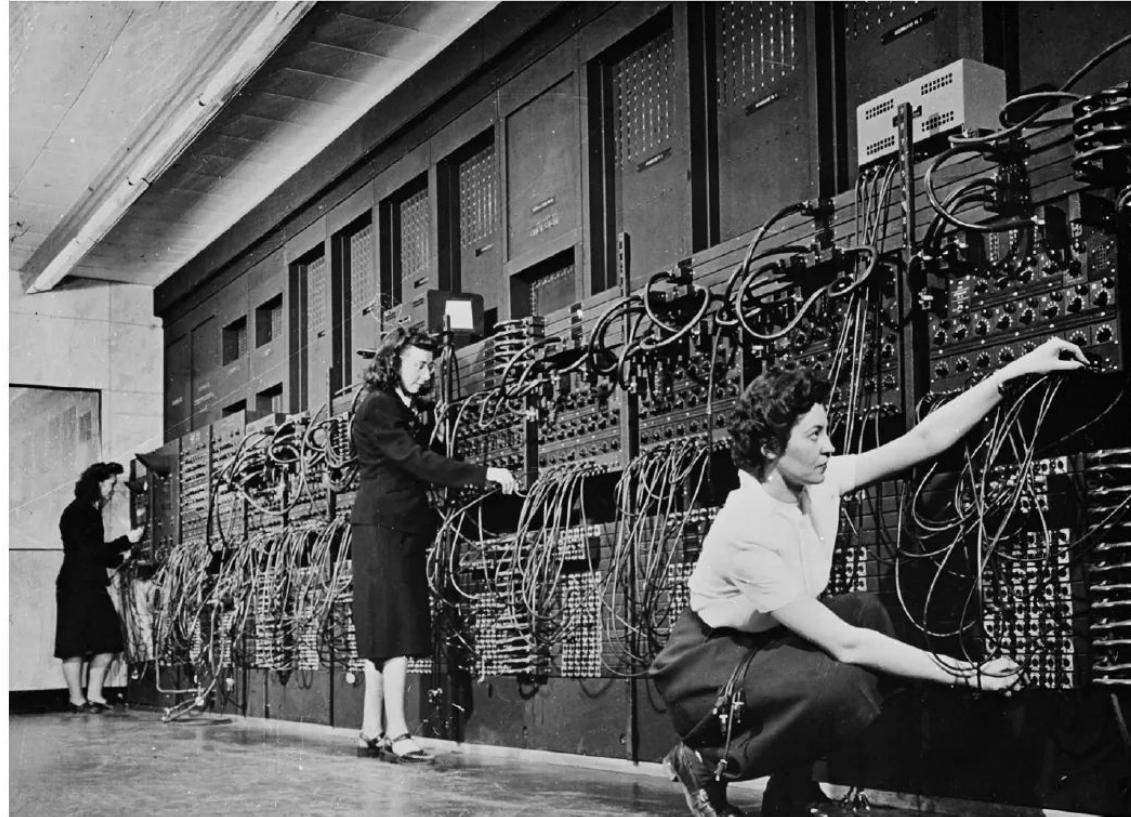
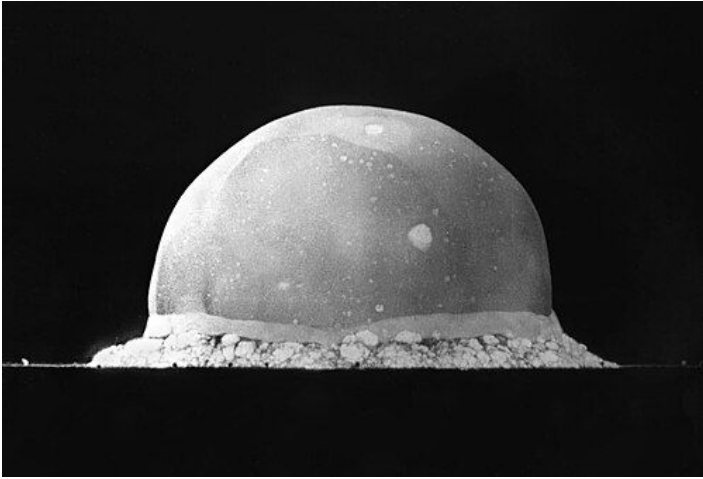
$$\{x_1, \dots, x_k, \dots, x_N\}$$



$$I \equiv \frac{1}{N} \sum_k f(x_k)$$

Motivación

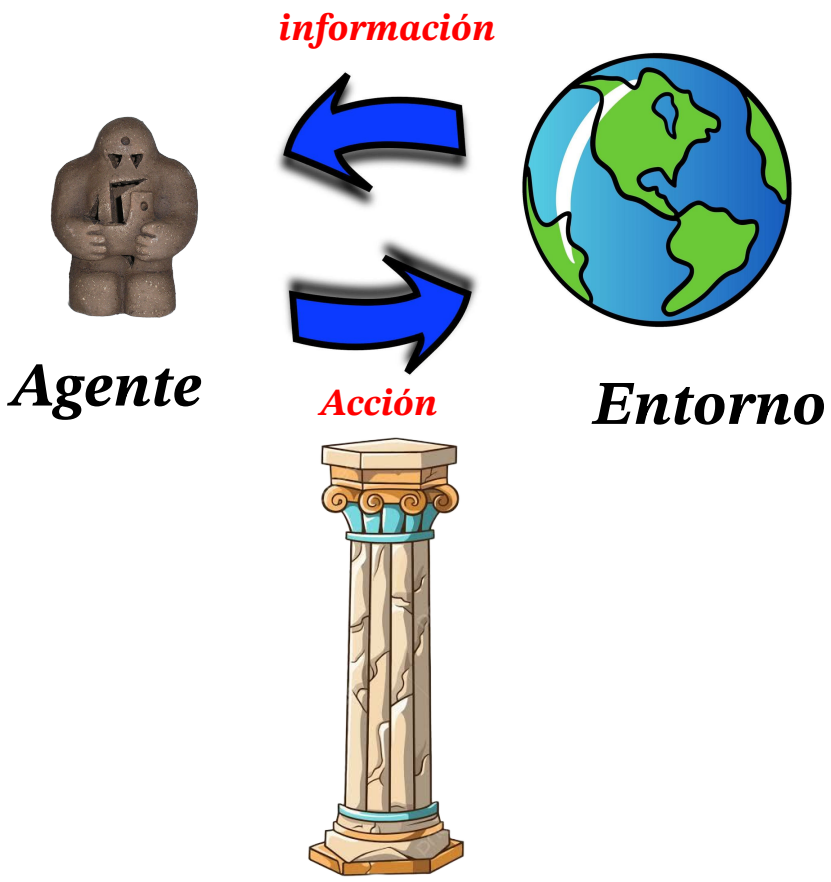
Dado el modelo... calculame algo... usá PC



[\[Historia del MC\]](#)

Monte Carlo en el contexto de RL

3 razones MC x RL



*Experiencia
directamente*

¿Se acuerdan cuando éramos jóvenes, Clase 1?

*El rabi le explicaba el universo //
esto es mi pie; esto el tuyo, esto la sogá //*

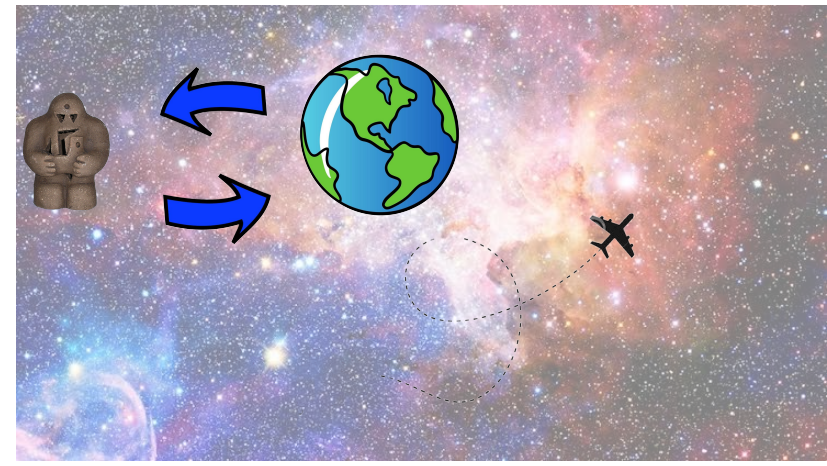
*y logró, al cabo de años, que el perverso //
barriera bien o mal la sinagoga*

¿De dónde sacamos $p(s'|s)$?

Una idea: hacer un modelito “físico” y reconstruir probs.

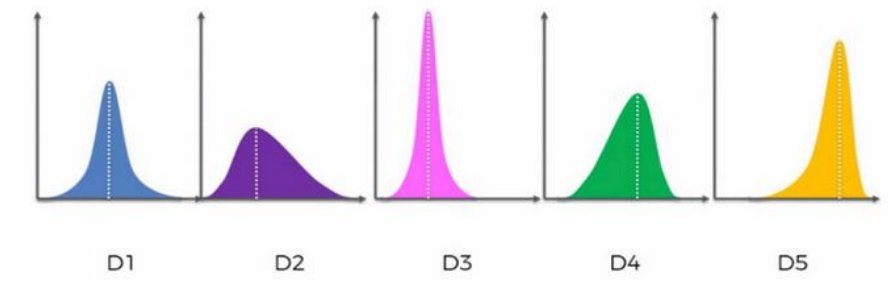
- Pero incluso reconstruyendo las transiciones $p(s'|s)$ de un “emulador”,
- cómo se traduce en “ $p(r|s,a)$ ”? **Difícil**

*Distribution vs.
sample models*



*No swepeear
todos los estados*

La clase 2 vimos...



- Estimamos $Q(a)$ para un bandit usando el promedio, [y las versiones incrementales y refinamientos]

$$q(a) = \mathbb{E}[R_t | A_t = a] \qquad \hat{q}(a) = \frac{\sum_{i=1}^{t-1} R_i \mathbb{I}_{A_i=a}}{\sum_{i=1}^{t-1} \mathbb{I}_{A_i=a}}$$

- Pero en el MDP, la acción nos va a llevar a otro estado!
Nos interesa estimar

$$v_{\pi}(s) = \mathbb{E}_{\pi}[G_t | S_t = s] = \mathbb{E}_{\pi}\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} | S_t = s\right]$$

La clase 4 vimos...

- *Policy prediction...* aprovechar la Bellman Equation para calcular predecir los value-function.

$$v_{\pi}(s) = \mathbb{E}_{\pi}[G_t | S_t = s] = \mathbb{E}_{\pi}\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} | S_t = s\right]$$

```
Loop:
  Δ ← 0
  Loop for each s ∈ S:
    v ← V(s)
    V(s) ← ∑a π(a|s) ∑s',r p(s',r|s,a) [r + γV(s')]
    Δ ← max(Δ, |v - V(s)|)
until Δ < θ
```



termination criteria
(si ya no cambia)

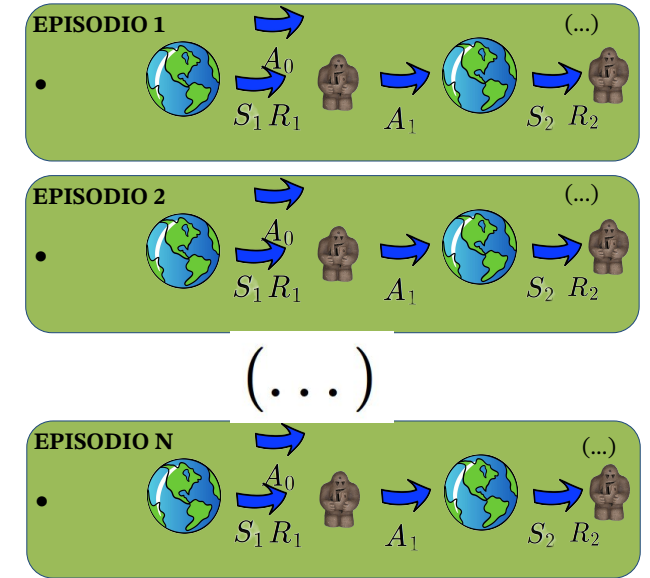


“bootstrapping”
(uso los v(s) que
voy modificando)

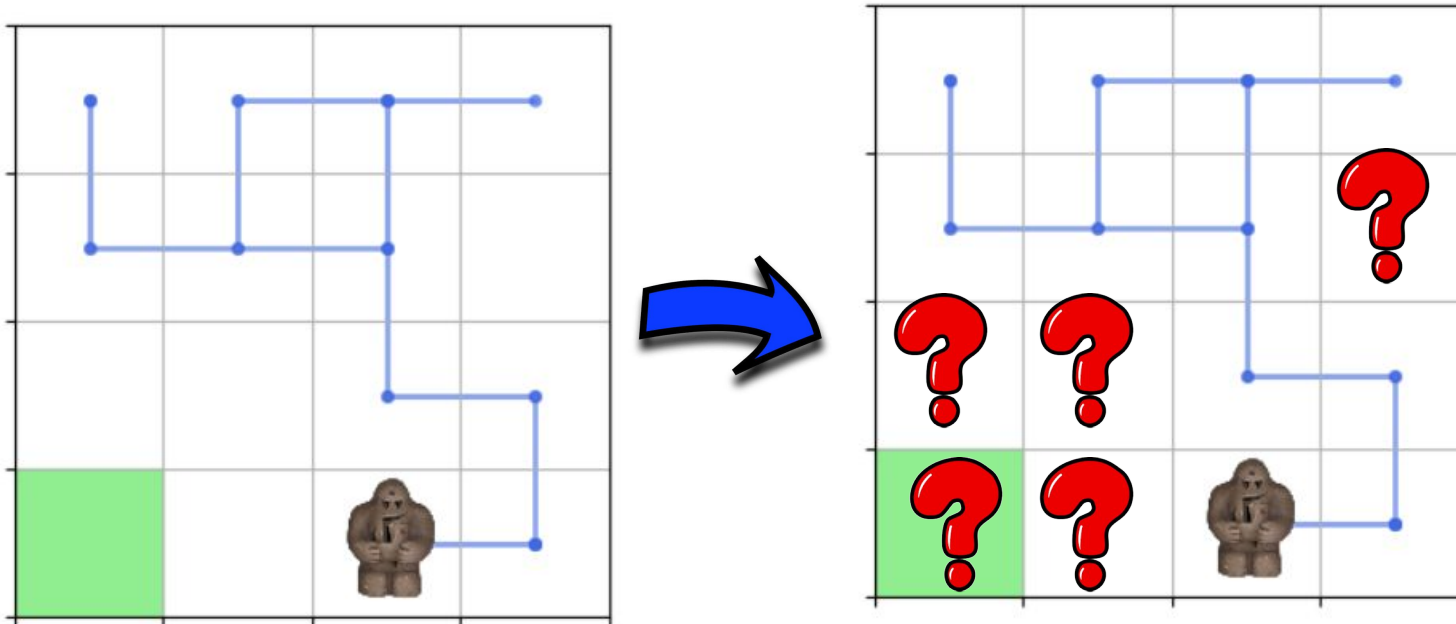


La clase 5... ¡oh sorpresa!

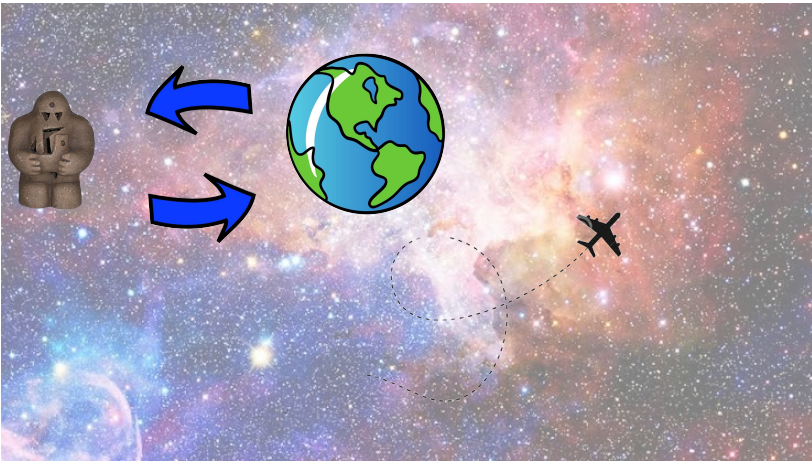
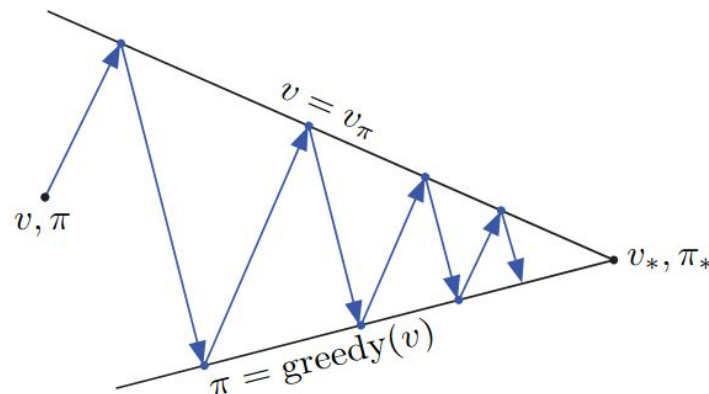
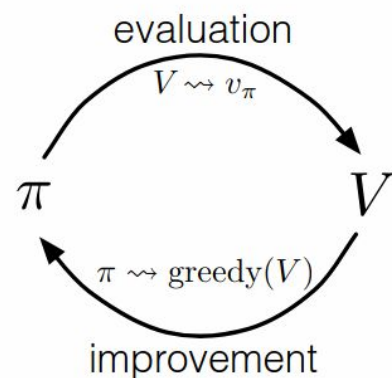
- *Obtener trayectoria (via experiencia),*
- *Calcular G_t*
- *Repetir para estimar $v_\pi(s) = \mathbb{E}_\pi[G_t | S_t = s]$*



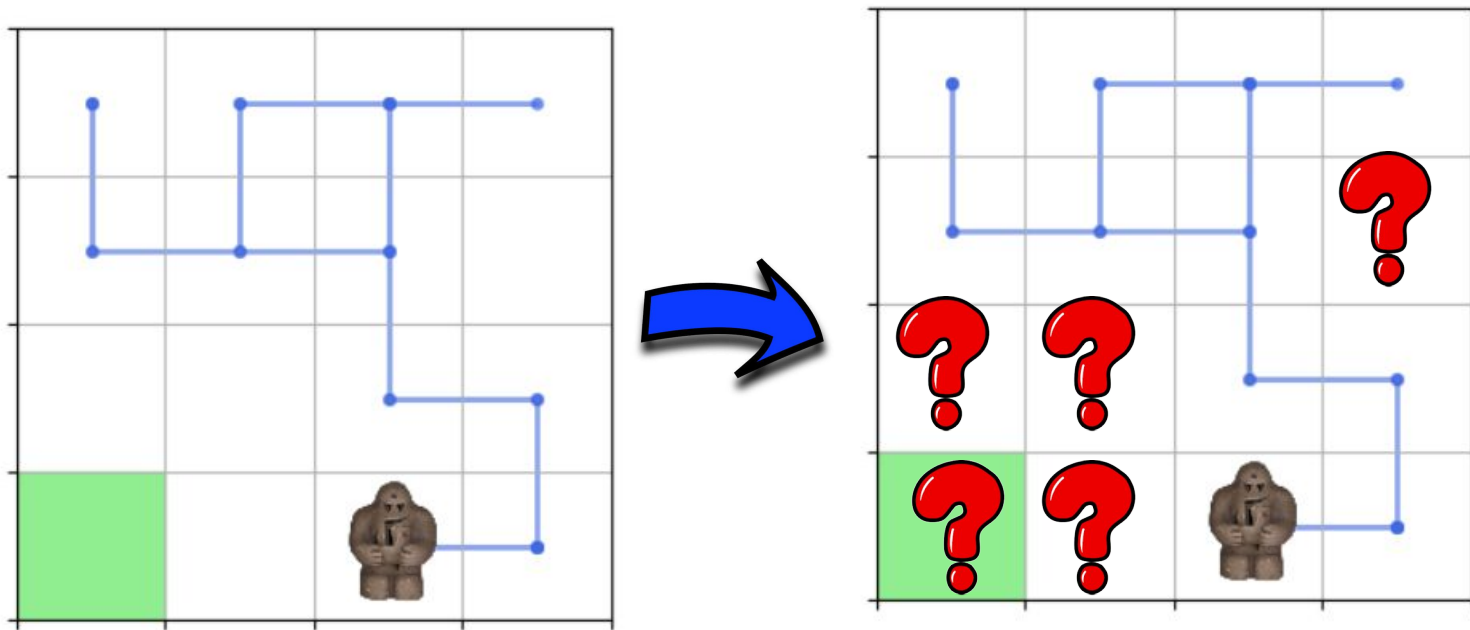
Dos fuentes de incerteza: $\hat{Q}_\pi(s, a)$?



La idea del GPI, *a full*



Dos fuentes de incerteza: $\hat{Q}_\pi(s, a)$?



Primera cuestión no trivial

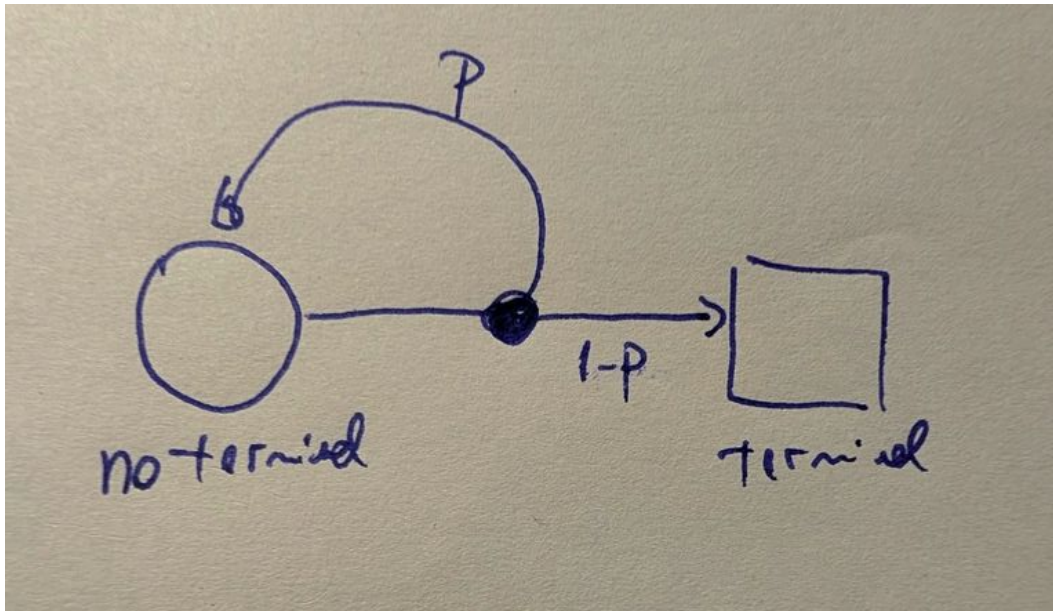
¿qué pasa si hay loops?

Ejercicio 5.5 S&B

Hay dos estados (T ; NT). El R_t es $+1$ para toda transición.
 $\gamma = 1$.

→ Si observamos un episodio que dura 10 steps...

...¿cómo estimamos G_t ?



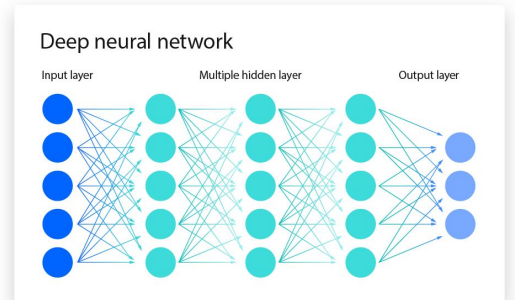
- 1) Cuál es el value function? $\rightarrow v(s) = 1/(1-p)$
- 2) First-visit: $V(s) = G_0 = 10$
- 3) Every-visit: $V(s) = (10 + 9 + \dots + 1)/10 = 5.5$

First-visit vs. every visit

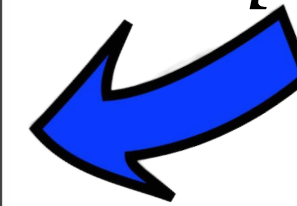
i.i.d



non-i.i.d



Llevar una cuenta de si pasamos por cada estado es “poco escalable”



[\[paper-singh-97\]](#)

First-visit MC prediction, for estimating $V \approx v_\pi$

Input: a policy π to be evaluated

Initialize:

$V(s) \in \mathbb{R}$, arbitrarily, for all $s \in \mathcal{S}$

$Returns(s) \leftarrow$ an empty list, for all $s \in \mathcal{S}$

Loop forever (for each episode):

Generate an episode following π : $S_0, A_0, R_1, S_1, A_1, R_2, \dots, S_{T-1}, A_{T-1}, R_T$

$G \leftarrow 0$

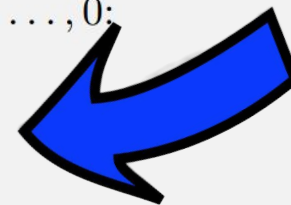
Loop for each step of episode, $t = T-1, T-2, \dots, 0$:

$G \leftarrow \gamma G + R_{t+1}$

Unless S_t appears in $S_0, S_1, \dots, S_{t-1}$:

Append G to $Returns(S_t)$

$V(S_t) \leftarrow \text{average}(Returns(S_t))$



*Every-visit
cuenta todo!*

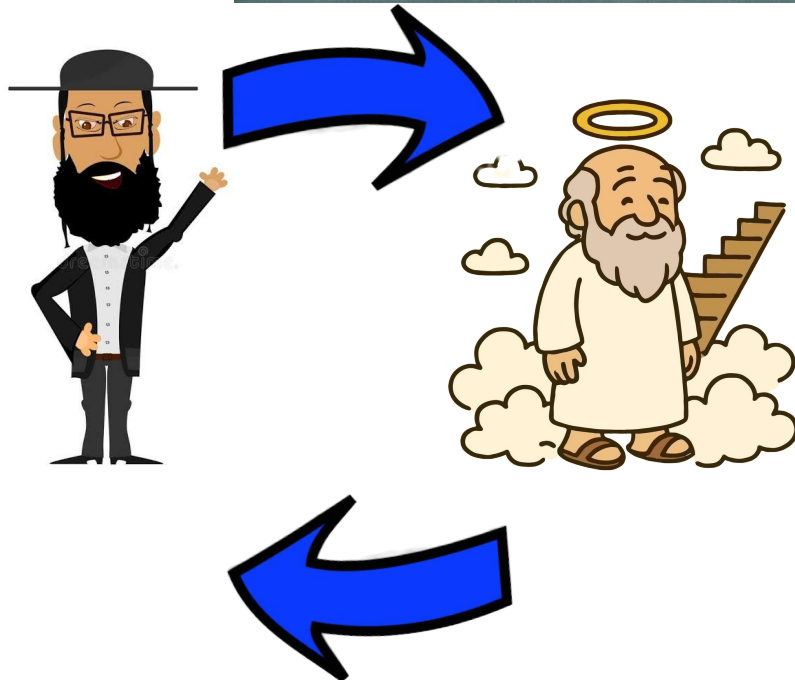
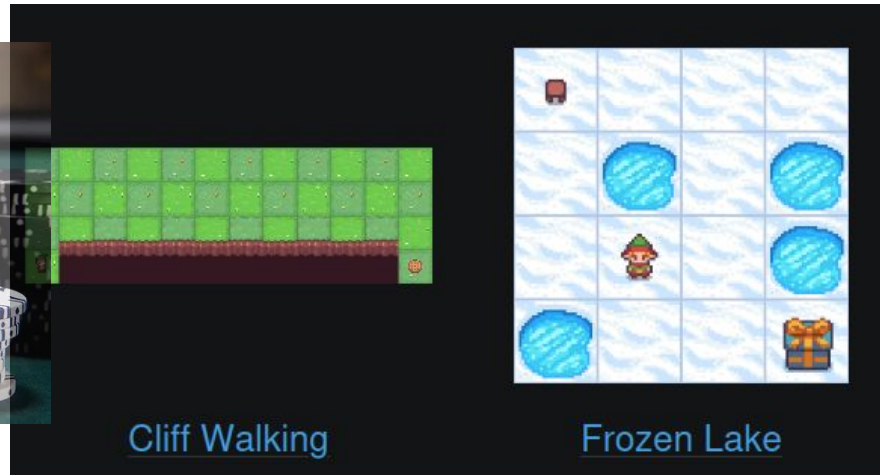
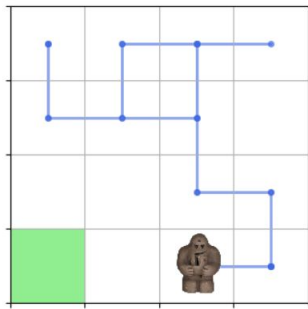
Las diferencias son drásticas para una realización (EV) es sobre muestras correlacionadas (dependientes), pero ambos estimadores convergen a $v(s)$

Memento Adorni



Entre dos aguas... ¿a qué jugamos?

El ejemplo de BlackJack está un poco entre medio...



Agente

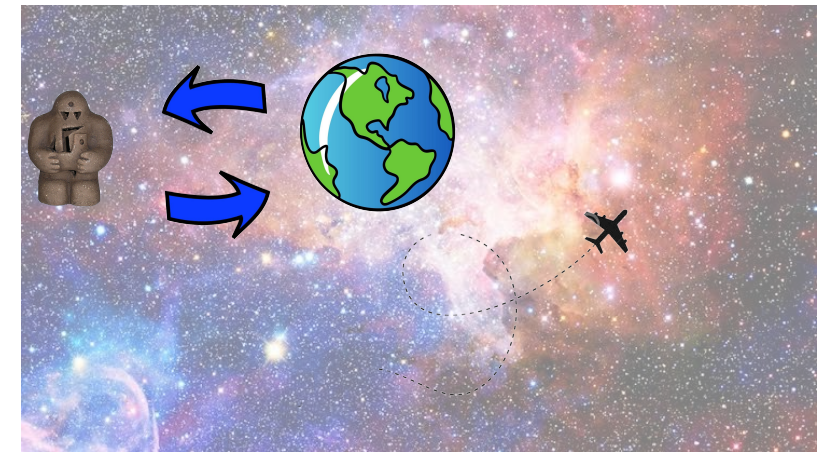
información



Acción



Entorno



BlackJack

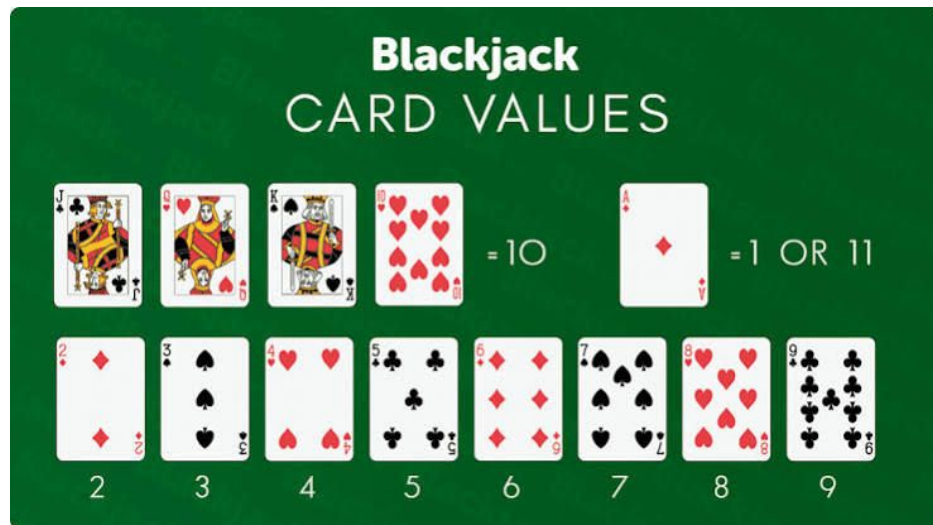
Vamos a ver un ejemplo de estimar los value functions en BJ
¿Por qué no hacerlo con Dynamic Programming?

- 1) *Ejemplo intermedio a “sin modelo”*
- 2) *DP impráctico: calcular $p(s'r|sa)$, aunque no imposible acá*
- 3) *Dada una estrategia de jugar al BlackJack, queremos predecir “cuán buena es”. La misma idea en mente en casos más complejos, donde DP es imposible por la dimensión*

BlackJack Reglas



Hay un **dealer** (repartidor)
y un **jugador**.
Empieza con dos cartas c/u



Valores:

$\{J, Q, K\} = 10$

$As = \{1, 11\}$

$\{2, 3, \dots, 9\}$ *el valor*

Estados

¿Estados?

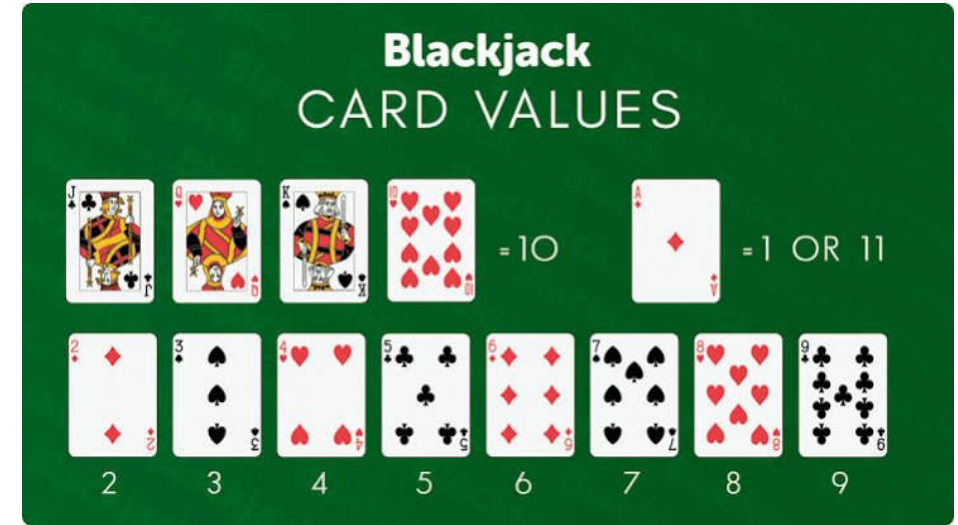
El dealer te reparte dos cartas y el tiene dos tmbn, muestra una.

→ {Mis cartas, La-del-dealer}

As usable o no:

{4, As} = 14 (as es usable)

{As, 9, 8} = 18 (ó 27, osea no usable)



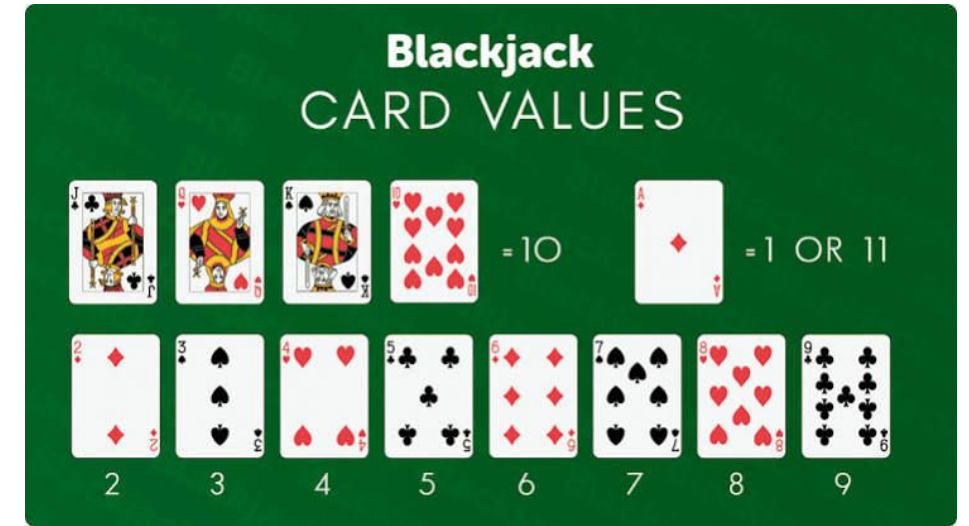
Hay un total de 200 estados:

- Dealer carta (As, 2, 3,...10) = 10 posibles
- Suma player = (12-21) = 10 posibles
- El As es usable o no = 2 posibles

Rewards

Goal: sumar 21 con las cartas, o acercarse tanto como sea posible, sin sobrepasarse.

$R = \{+1, -1, 0\}$
= {gana, pierde, empata}



Ejemplo, si tengo el {J,As} gano “natural”
(salvo que dealer tmbn tenga --> empate = draw)

Acciones

Acciones:

$\{\underline{hit} = \text{pedir carta},$

$\underline{stick} = \text{parar}\}$

Otra terminología: bust = sobrepasarse de 21 = perder

dealer ---> *fixed strategy ...*

ej. “stick” si tiene 17 o más, hit else



¿¿¿Cuán buena es una estrategia que tengamos?? MonteCarlo!

Ejemplo estrategia: stick si $\text{sum} = \{20, 21\}$, hit sino

BlackJack gym: MC prediction

Fijate que First-visit = Every-visit (*porque no hay bucles!*)
Gym devuelve estados (suma, dealer, As)

```
import gymnasium as gym
import matplotlib.pyplot as plt
import numpy as np

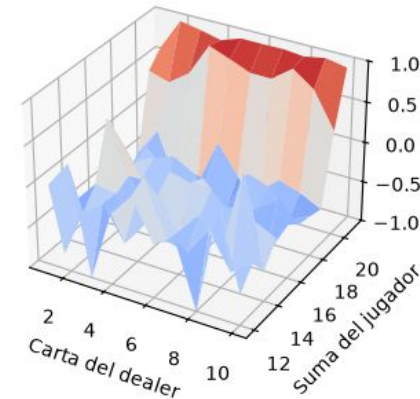
env = gym.make("Blackjack-v1", sab=True) # sab=True: reglas exactas de Sutton & Barto
print(env.observation_space)           # (suma del jugador, carta visible del dealer, ¿as usable?)
print(env.action_space)                # 0 = stick (plantarse), 1 = hit (pedir)
print(hasattr(env.unwrapped, "P"))     # False: acá no hay p(s',r|s,a) escrita a mano

# %% La policy a evaluar
def policy(obs):
    player_sum, dealer_card, usable_ace = obs
    return 0 if player_sum >= 20 else 1

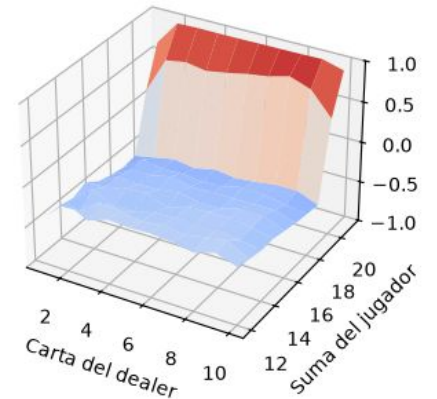
# %% Una mano
obs, info = env.reset(seed=18)
done = False
while not done:
    a = policy(obs)
    obs_next, r, terminated, truncated, info = env.step(a)
    print(obs, "hit " if a else "stick", "->", obs_next, "reward", r)
    obs, done = obs_next, terminated or truncated

# %% First-visit MC prediction (acá = every-visit: no hay loops)
```

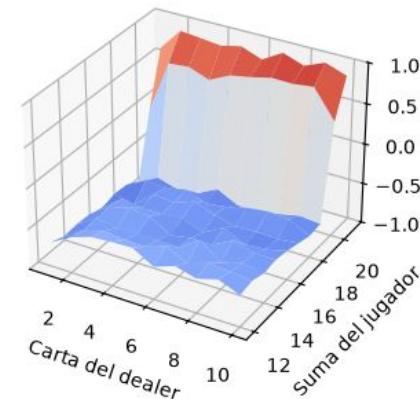
10.000 episodios — con as usable



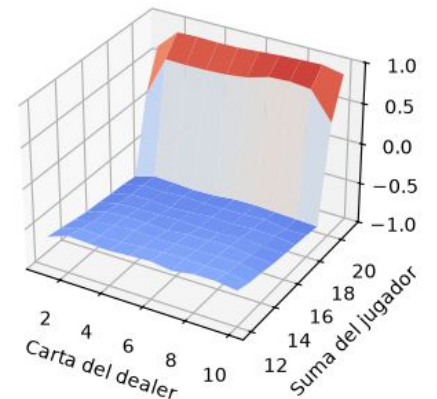
500.000 episodios — con as usable



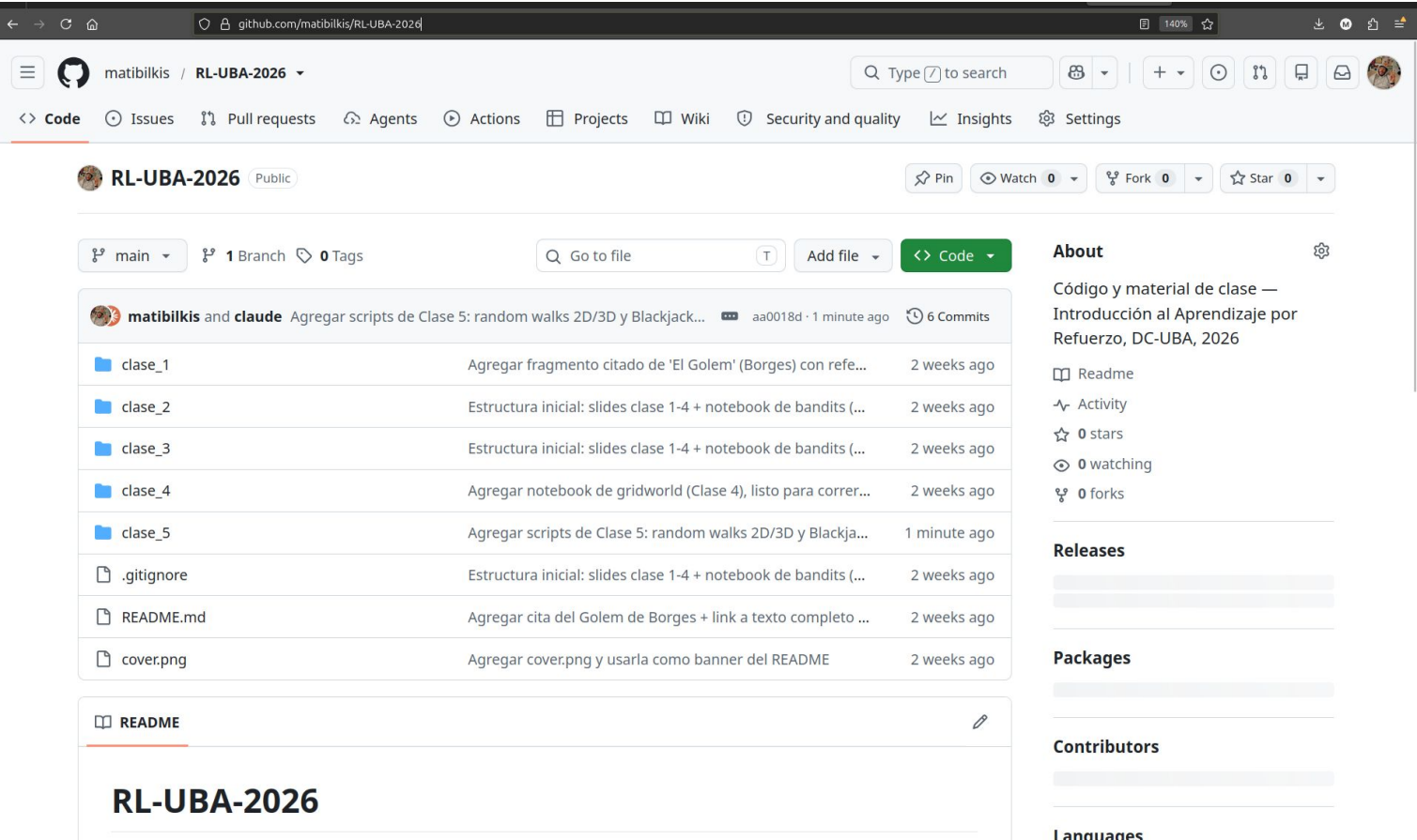
10.000 episodios — sin as usable



500.000 episodios — sin as usable



BlackJack gym



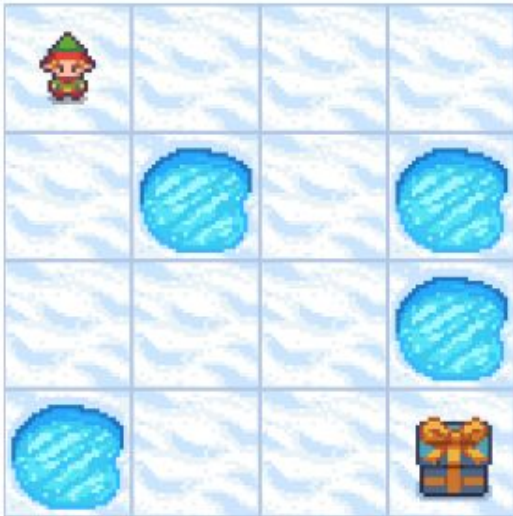
[LINK](#)



*el loop / código
es bastante genérico!*

Más de gym

FrozenLake-v1



FrozenLake-v1 8x8



```
4_taxi_frozenlake.py -- ~/reluba -- Pulsar
Help
6_random_walks.py 4_taxi_frozenlake.py 2_blackjack_prediction.py blackjack.py
1 # %% FrozenLake y Taxi: lo básico de cada entorno
2 import gymnasium as gym
3 import matplotlib.pyplot as plt
4 import numpy as np
5
6 def random_episode(env):
7     obs, info = env.reset()
8     G, T, done = 0, 0, False
9     while not done:
10         obs, r, terminated, truncated, info = env.step(env.action_space.sample())
11         G, T = G + r, T + 1
12         done = terminated or truncated
13     return G, T, truncated
14
15 # %% FrozenLake: S inicio, F hielo, H agujero (termina), G meta (reward 1). Todo lo demás: reward 0
16 env = gym.make("FrozenLake-v1", map_name="4x4", is_slippery=True, render_mode="rgb_array")
17 print(env.observation_space, env.action_space) # Discrete(16) Discrete(4): 0 ←, 1 ↓, 2 →, 3 ↑
18 print("time limit:", env.spec.max_episode_steps)
19 obs, info = env.reset(seed=0)
20 plt.imshow(env.render())
```

<matplotlib.image.AxesImage at 0x7222d6914f20>

A 4x4 grid representing the FrozenLake-v1 environment, rendered with axes. The x and y axes range from 0 to 250. The top-left cell contains a small character (the agent). The bottom-right cell contains a gift box (the goal). The other cells are either empty or contain a blue ice patch, indicating slippery terrain.

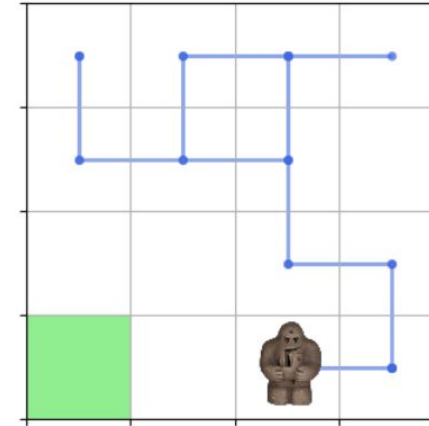
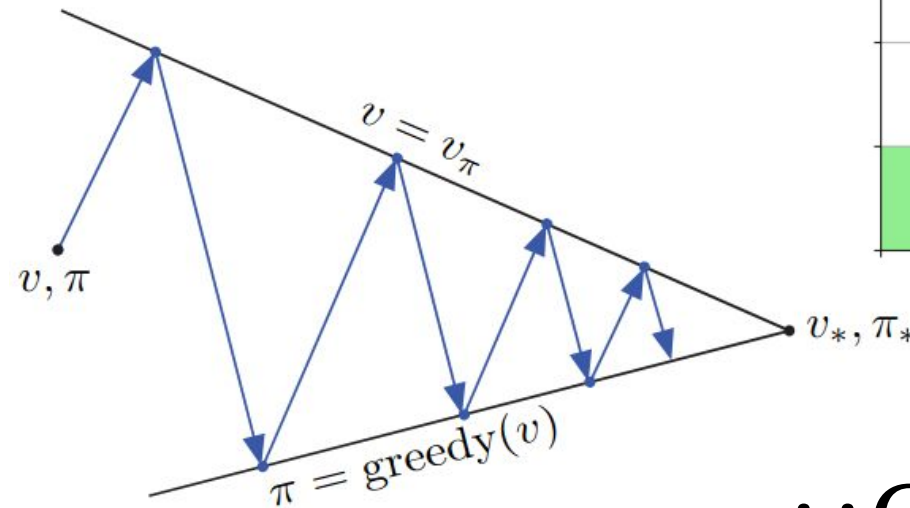
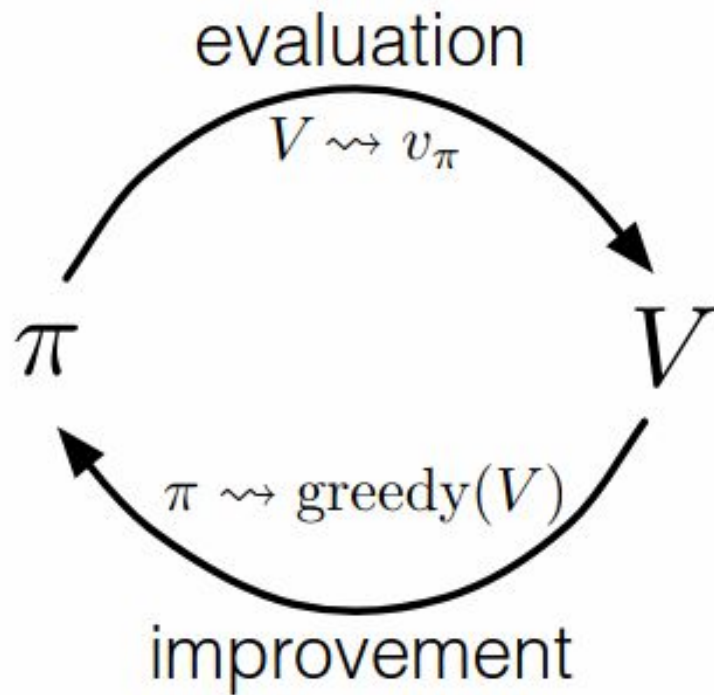
Gym Farama

*Tarea: revisar & describir
el taxi & Cliff Walking*

**Tomemos el control
(Monte-Carlo control)**

Nada nuevo bajo el Sol, ¿no?

Siguiendo la lógica que introdujimos en DP (Clase 4)...
podemos hacer “value iteration”, pero c/ MC & Q(s,a)



¿¿Cómo garantizamos
la exploración??

Exploring starts (init random)

Monte Carlo ES (Exploring Starts), for estimating $\pi \approx \pi_*$

Initialize:

$\pi(s) \in \mathcal{A}(s)$ (arbitrarily), for all $s \in \mathcal{S}$

$Q(s, a) \in \mathbb{R}$ (arbitrarily), for all $s \in \mathcal{S}, a \in \mathcal{A}(s)$

$Returns(s, a) \leftarrow$ empty list, for all $s \in \mathcal{S}, a \in \mathcal{A}(s)$

Loop forever (for each episode):

Choose $S_0 \in \mathcal{S}, A_0 \in \mathcal{A}(S_0)$ randomly such that all pairs have probability > 0

Generate an episode from S_0, A_0 , following π : $S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T$

$G \leftarrow 0$

Loop for each step of episode, $t = T-1, T-2, \dots, 0$:

$G \leftarrow \gamma G + R_{t+1}$

Unless the pair S_t, A_t appears in $S_0, A_0, S_1, A_1, \dots, S_{t-1}, A_{t-1}$:

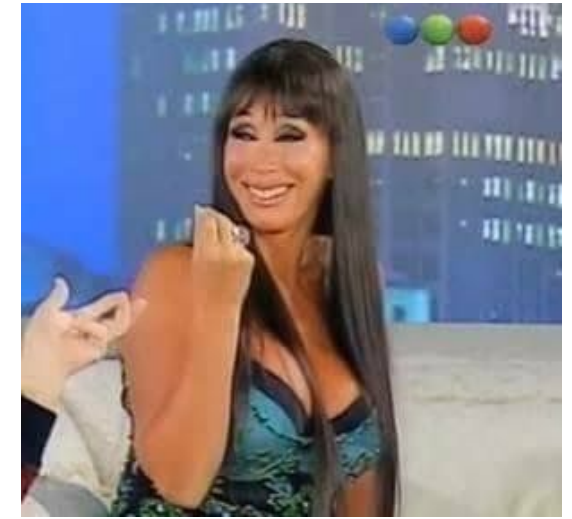
Append G to $Returns(S_t, A_t)$

$Q(S_t, A_t) \leftarrow \text{average}(Returns(S_t, A_t))$

$\pi(S_t) \leftarrow \operatorname{argmax}_a Q(S_t, a)$

Exploring Starts vs. MDPs

No es muy realista que se inicializa random para visitar todos los pares $\{(s,a)\}$ en gral...



On-policy

uso la misma policy
para evaluar & actuar



Off-policy

evalúo una (π)
mientras sigo otra (b)

... on-policy... debería funcionar?

On-policy control con ε -greedy... “*Discusión teórica*”

Teóricamente sí (Sec. 5.4 S&B), pero *as good as* ε -greedy

On-policy first-visit MC control (for ε -soft policies), estimates $\pi \approx \pi_*$

Algorithm parameter: small $\varepsilon > 0$

Initialize:

$\pi \leftarrow$ an arbitrary ε -soft policy

$Q(s, a) \in \mathbb{R}$ (arbitrarily), for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$

$Returns(s, a) \leftarrow$ empty list, for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$

Repeat forever (for each episode):

Generate an episode following π : $S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T$

$G \leftarrow 0$

Loop for each step of episode, $t = T-1, T-2, \dots, 0$:

$G \leftarrow \gamma G + R_{t+1}$

Unless the pair S_t, A_t appears in $S_0, A_0, S_1, A_1, \dots, S_{t-1}, A_{t-1}$:

Append G to $Returns(S_t, A_t)$

$Q(S_t, A_t) \leftarrow \text{average}(Returns(S_t, A_t))$

$A^* \leftarrow \arg\max_a Q(S_t, a)$ (with ties broken arbitrarily)

For all $a \in \mathcal{A}(S_t)$:

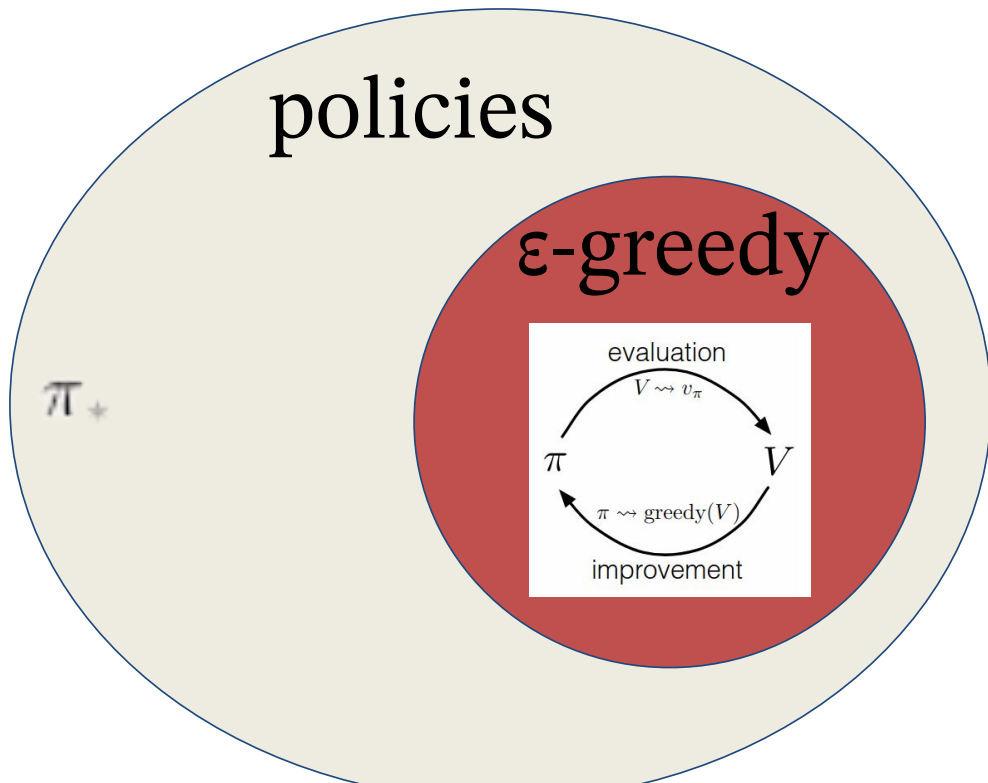
$$\pi(a|S_t) \leftarrow \begin{cases} 1 - \varepsilon + \varepsilon/|\mathcal{A}(S_t)| & \text{if } a = A^* \\ \varepsilon/|\mathcal{A}(S_t)| & \text{if } a \neq A^* \end{cases}$$



... on-policy... debería funcionar?

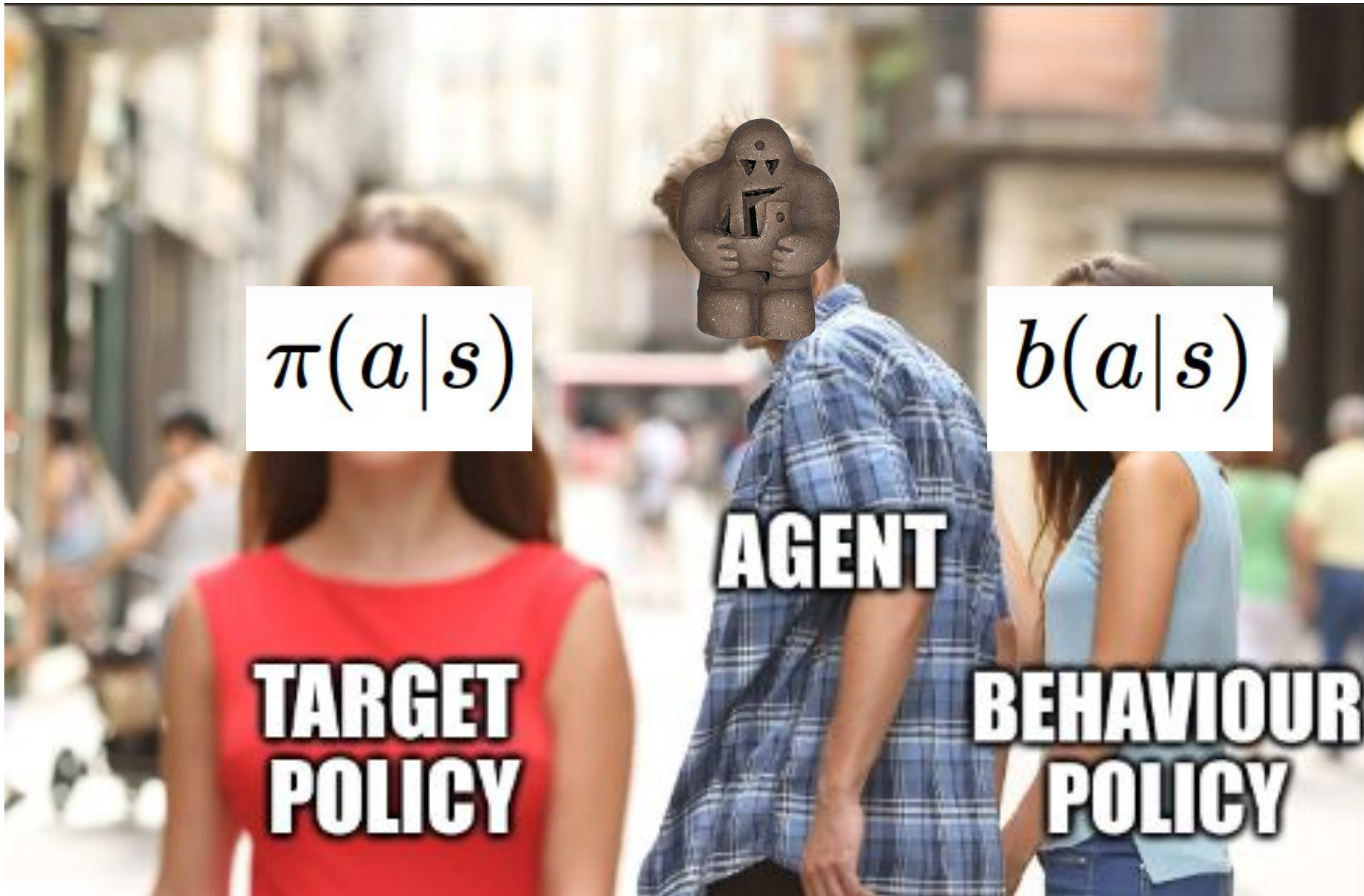
A quien le interese leerlo, el truco más sofisticado que usan

$$\sum_i w_i x_i \leq \max_i x_i \quad w_i \geq 0, \sum_i w_i = 1$$



→ policy iteration funciona sobre ϵ -soft *policies*

Off-policy



Off-policy: importance sampling

¿Cómo aprendo francés si me explican en inglés?
¿Cómo aprende el Golem a hablar si le enseñan a barrer?
...¿se entiende?

***Necesitamos** la probabilidad relativa de la experiencia bajo una y otra política... eso es el importance sampling (ratio...).*

Cómo construimos el ratio no es trivial, y cambia según la idea
[hay muchas aplicaciones, beyond RL]

***off policy** → Q-learning (clase 6), **IS** → PPO (clase 11-12)*

Off-policy

Behavior policy b : la que genera los datos (explora).

Target policy π : la que quiero aprender (e.g. greedy).
Necesitamos pedir - para que haya solapamiento- que:

$$\text{Coverage: } \pi(a|s) > 0 \implies b(a|s) > 0$$



Off-policy: importance sampling

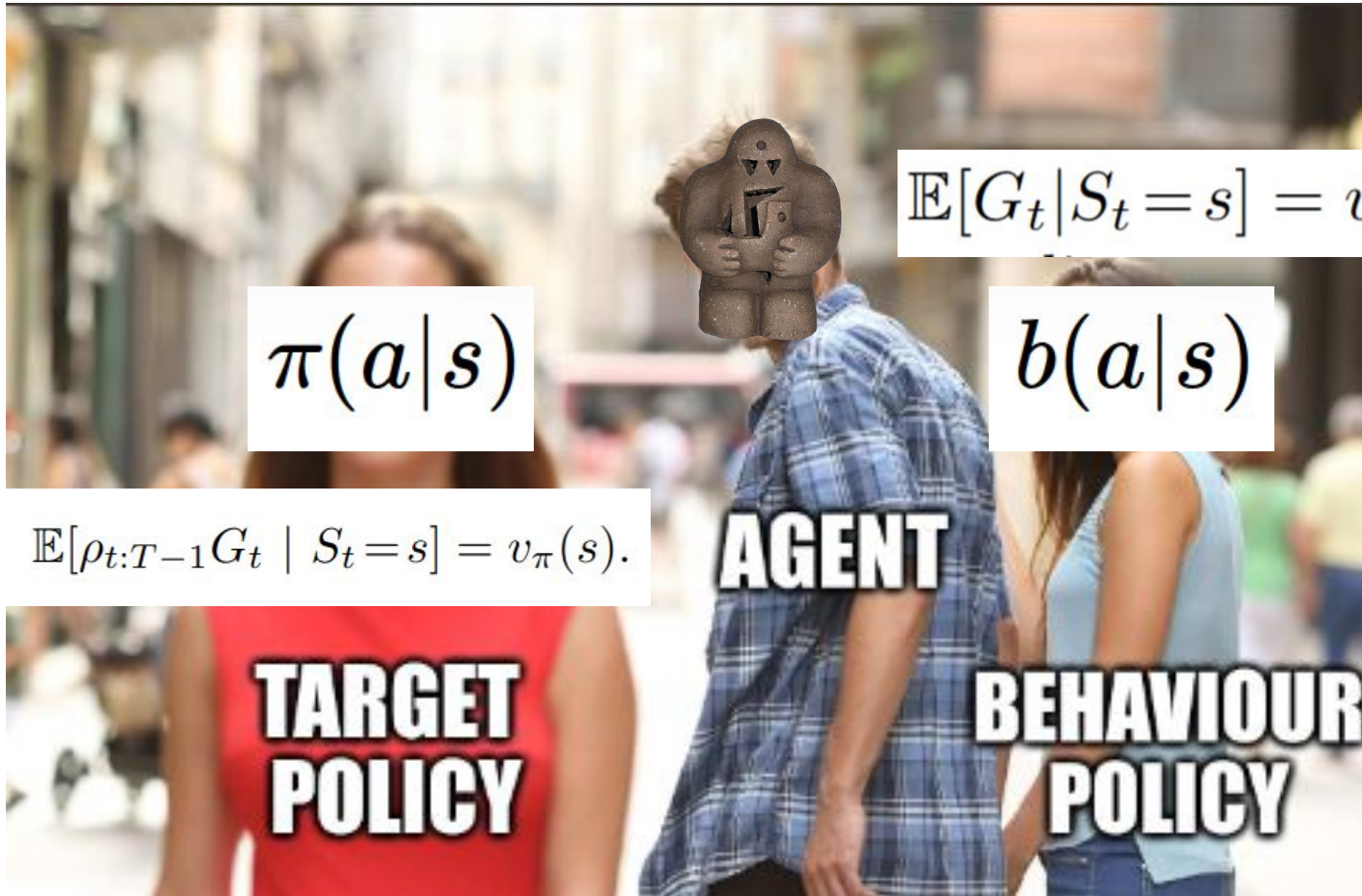
Dada una trayectoria... $A_t, S_{t+1}, A_{t+1}, \dots, S_T,$

Su probabilidad es $\Pr\{A_t, S_{t+1}, A_{t+1}, \dots, S_T \mid S_t, A_{t:T-1} \sim \pi\}$
 $= \pi(A_t|S_t)p(S_{t+1}|S_t, A_t)\pi(A_{t+1}|S_{t+1}) \cdots p(S_T|S_{T-1}, A_{T-1})$
 $= \prod_{k=t}^{T-1} \pi(A_k|S_k)p(S_{k+1}|S_k, A_k),$

La probabilidad relativa sobre una policy u otra es el IS ratio

$$\rho_{t:T-1} \doteq \frac{\prod_{k=t}^{T-1} \pi(A_k|S_k)p(S_{k+1}|S_k, A_k)}{\prod_{k=t}^{T-1} b(A_k|S_k)p(S_{k+1}|S_k, A_k)} = \prod_{k=t}^{T-1} \frac{\pi(A_k|S_k)}{b(A_k|S_k)}.$$

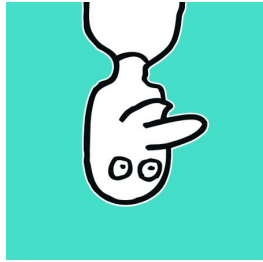
Off-policy: importance sampling



Off-policy: importance sampling

Acá se pone medio delicado, porque puedo estimarlo de (al menos) dos maneras diferentes

$$V(s) \doteq \frac{\sum_{t \in \mathcal{T}(s)} \rho_{t:T(t)-1} G_t}{|\mathcal{T}(s)|}.$$



Ordinary IS

$$V(s) \doteq \frac{\sum_{t \in \mathcal{T}(s)} \rho_{t:T(t)-1} G_t}{\sum_{t \in \mathcal{T}(s)} \rho_{t:T(t)-1}},$$



weighted IS

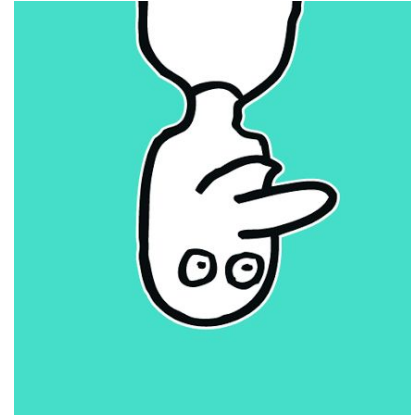
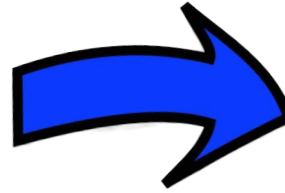
Off-policy: importance sampling (IS)



On-policy

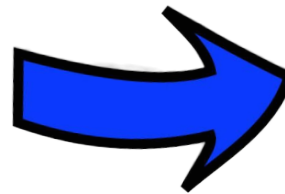


Off-policy



ordinary (IS)

- unbiased
- VARIANCE



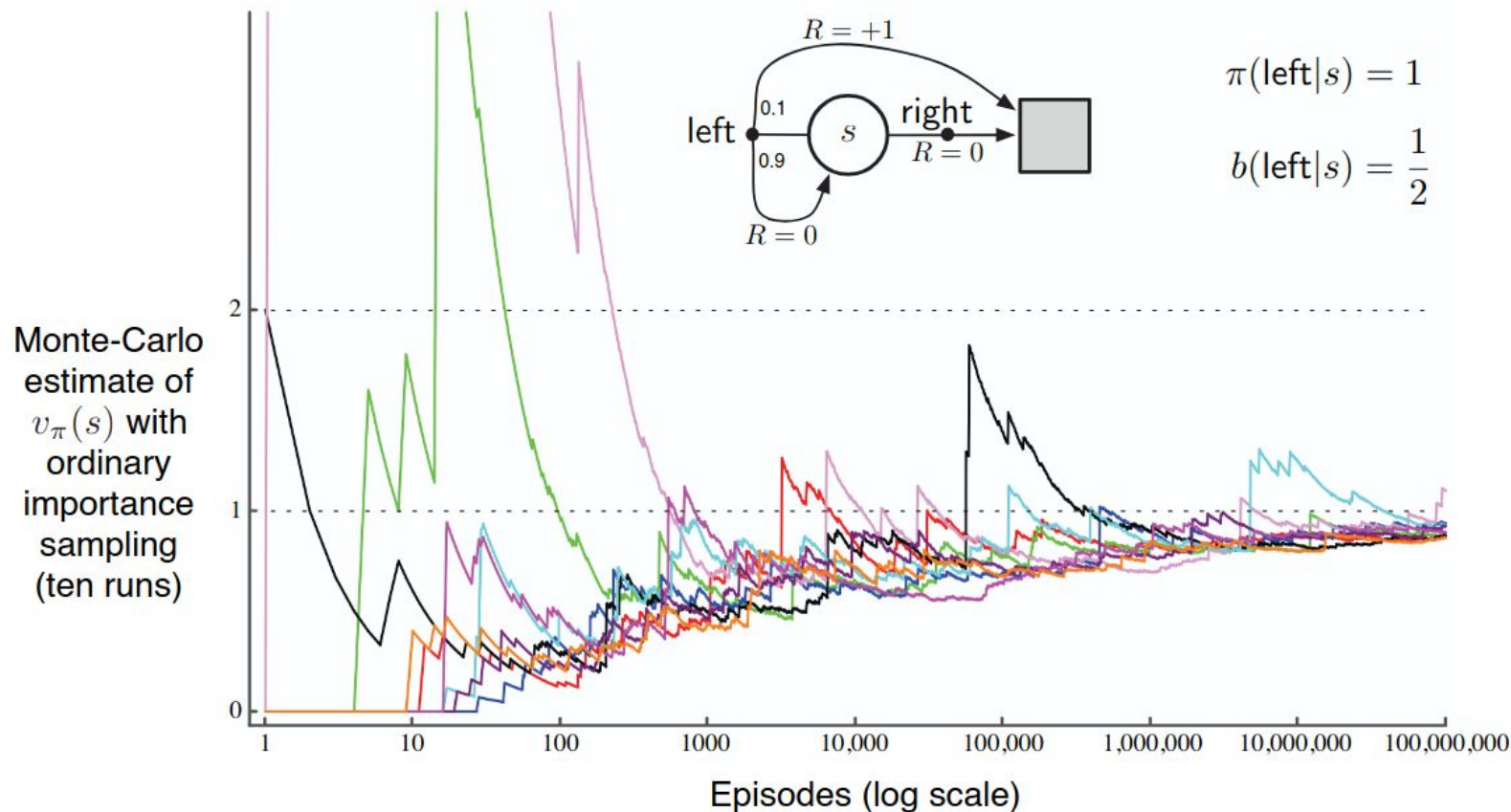
weighted (IS)

- biased
- variance
- meyor converg..

First-visit	vs.	every visit
✗ Non-IID		✓ IID

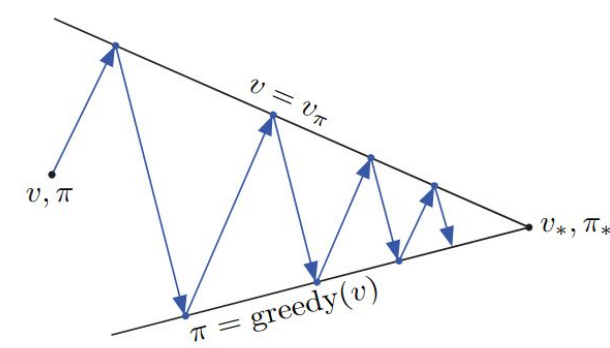
Ejemplo 5.5 Sutton

*Ordinary IS no está sesgado, pero puede tener $Var = \infty$
Weighted IS está sesgado, pero funciona bárbaro :-)*



**¿Perdimos el control?
(Monte-Carlo control)**

Off policy MC control



Juntando las ideas... el “target” es la greedy policy, y la incrementación incremental que vimos en Clase 2...

Off-policy MC control, for estimating $\pi \approx \pi_*$

Initialize, for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$:

$Q(s, a) \in \mathbb{R}$ (arbitrarily)

$C(s, a) \leftarrow 0$

$\pi(s) \leftarrow \operatorname{argmax}_a Q(s, a)$ (with ties broken consistently)

Loop forever (for each episode):

$b \leftarrow$ any soft policy

Generate an episode using b : $S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T$

$G \leftarrow 0$

$W \leftarrow 1$

Loop for each step of episode, $t = T-1, T-2, \dots, 0$:

$G \leftarrow \gamma G + R_{t+1}$

$C(S_t, A_t) \leftarrow C(S_t, A_t) + W$

$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \frac{W}{C(S_t, A_t)} [G - Q(S_t, A_t)]$

$\pi(S_t) \leftarrow \operatorname{argmax}_a Q(S_t, a)$ (with ties broken consistently)

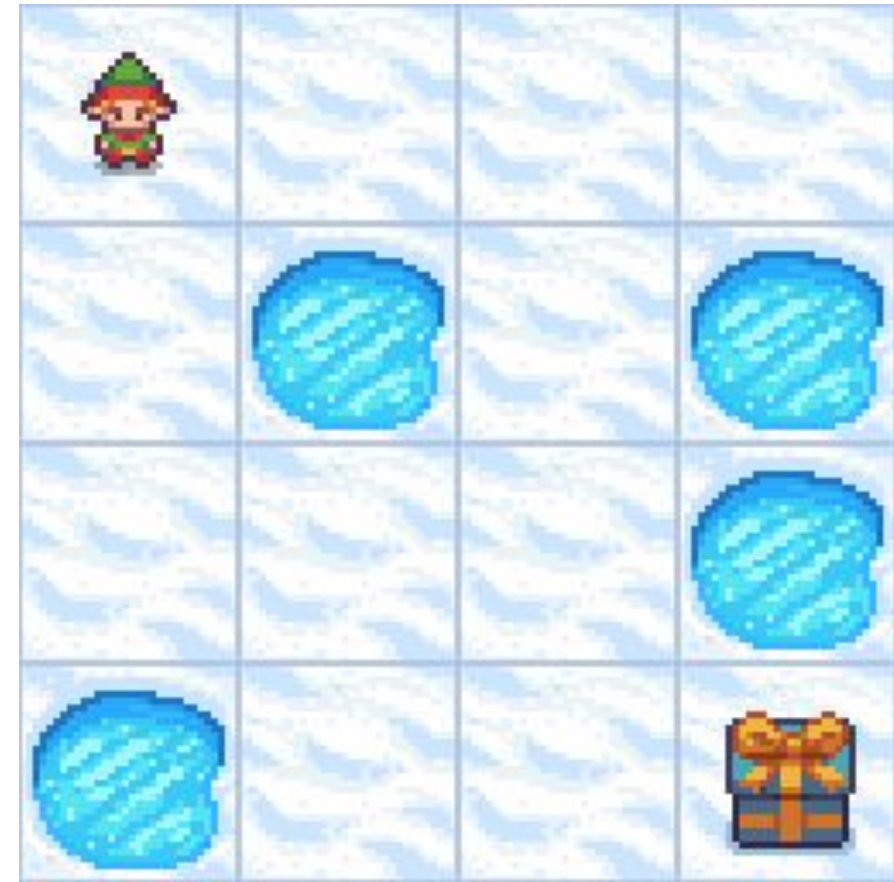
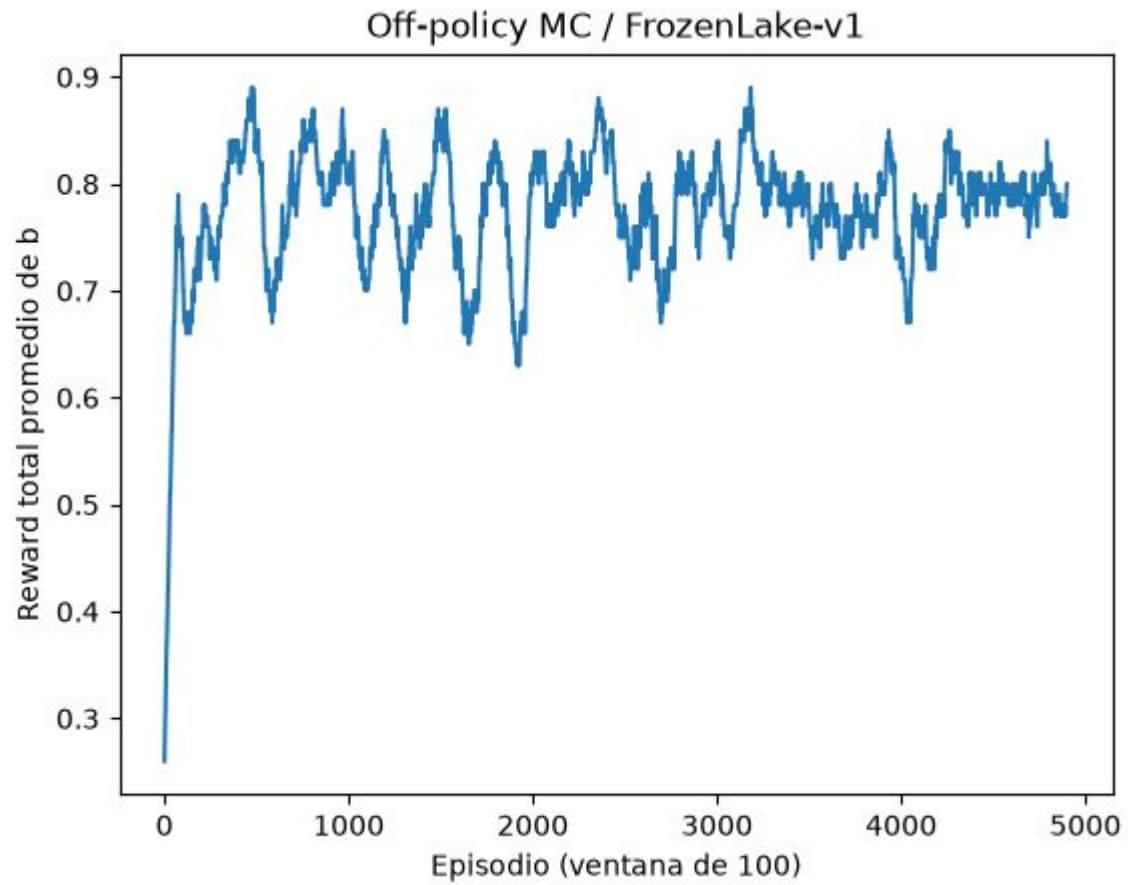
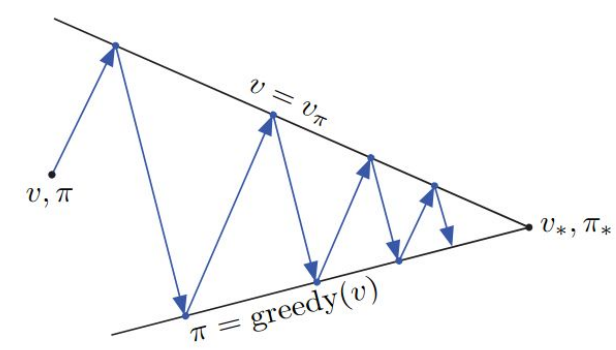
If $A_t \neq \pi(S_t)$ then exit inner Loop (proceed to next episode)

$W \leftarrow W \frac{1}{b(A_t|S_t)}$

$$\hat{q}_{n+1}(a) = \frac{R_1 + \dots + R_n}{n}$$

$$\hat{q}_{n+1}(a) = \hat{q}_n(a) + \frac{1}{n} (R_n - \hat{q}_n(a))$$

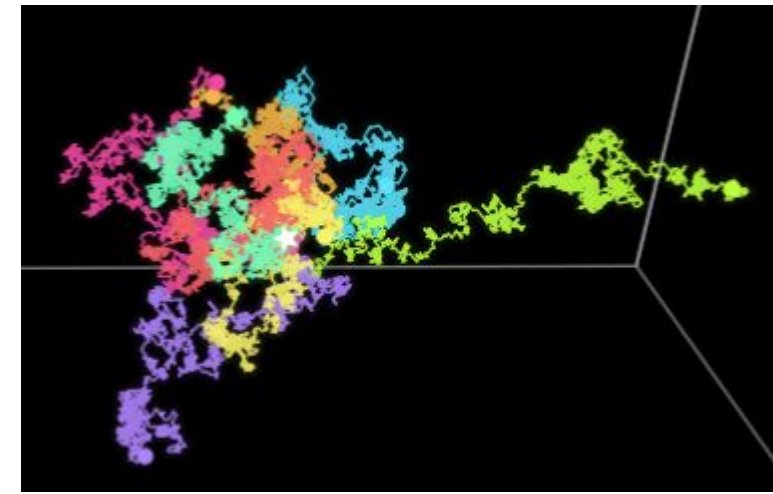
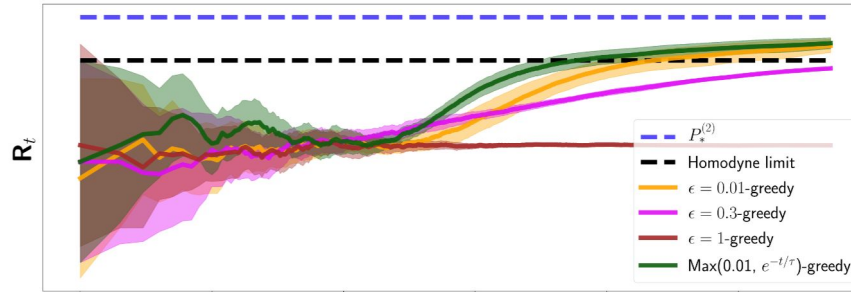
Off policy MC control



Cosas malas del MC

- *Hay que esperar a tener toda la trayectoria para poder “aprender”... MUCHA VARIANZA!*
- *...no “bootstrapea” →*  *(no aprovecha $V(s')$ para corregir $V(s)$)*
- *MC control → Dudosa convergencia*
- *No sirve para tareas continuas*
- *Tampoco puede mezclar G_t obtenidos de distintas trayectorias (más que en el promedio)*

Tarea



- Leer S&B Capítulo 5 (MC). Saltar (los *)
- Comparar ON-policy & OFF-policy sobre N agentes para el Frozen-Lake, las learning curves. schedule ep
- Opcional: graficar random walks, que siempre es lindo! en 2D, 3D.
- *revisar & describir el [taxi & Cliff Walking](#)*
Procedimiento, lo mismo que las clases pasada!)